

UNIT



Regular Expressions, Pumping Lemma for Regular Languages and Context-free Grammars

SYLLABUS

Regular Expressions: Finite Automata and Regular Expressions, Applications of Regular Expressions, Algebraic Laws for Regular Expressions, Conversion of Finite Automata to Regular Expressions.

Pumping Lemma for Regular Languages: Statement of the Pumping lemma, Applications of the Pumping Lemma.

Context-Free Grammars: Definition of Context-Free Grammars, Derivations Using a Grammar, Leftmost and Rightmost Derivations, The Language of a Grammar, Parse Trees, Ambiguity in Grammars and Languages.

LEARNING OBJECTIVES

- ✓ Definition, Properties and Rules of Regular Expression
- ✓ Conversion of Regular Expressions to Finite Automata
- ✓ Applications and Algebraic Laws of Regular Expressions
- ✓ Conversion of Finite Automata to Regular Expressions
- ✓ Proof and Applications of Pumping Lemma
- ✓ Definition of Context-Free Grammar
- ✓ Concept of Parse Tree and its Types
- ✓ Checking Ambiguity of a Grammar

INTRODUCTION

A Regular Expression is a concise notation for denoting regular sets. It describes the language accepted by a finite automata. Regular language is a formal language generated from regular sets over input alphabet S . Let S be a finite alphabet then a class containing set of strings over S is called regular sets. Regular expressions and finite automata are said to be equivalent, because the languages which are generated by regular expressions are also accepted by finite automata and vice-versa.

Pumping Lemma is used to prove that certain languages are not regular. It is also used to determine whether the language accepted by a finite automata is finite or infinite.

A Context Free Grammar (CFG) describes the programming language constructs. It consists of terminals, non-terminals, a start symbol and productions. Whereas, derivation is a method of replacing the non-terminal symbol present on RHS of a production rule with a terminal input. A tree representation for the derivation of the given production rules for a CFG is called a derivation tree/parse tree. Moreover, the concept of ambiguity arises if the given Grammar (G) produces more than one derivation tree for a given input string (w).

2.1 REGULAR EXPRESSIONS

Q1. Define Regular expression. Explain about the properties of regular expressions.

Answer : (Dec.-18(R16), Q4(a) | April-18(R16), Q4(a))

Regular Expression

A regular expression is a concise notation for denoting regular sets. Regular expressions describe the language accepted by a finite automata.

Let Σ be a finite alphabet. Then the regular expressions over Σ denoting the regular sets are recursively defined as follows,

- (a) ϕ is a regular expression denoting the regular set $\{\phi\}$.
- (b) ϵ is a regular expression denoting the regular set $\{\epsilon\}$.
- (c) a is a regular expression denoting the regular set $\{a\}$.

If P and Q are regular sets of languages L_1 and L_2 , then

- ❖ $(P + Q)$ is a regular expression denoting the set $P \cup Q$.
- ❖ PQ is a regular expression denoting the set PQ .
- ❖ P^* is a regular expression denoting the set P^* .

Example

$0(0+1)^*1$ is a regular expression denoting the set of all strings over $\{0, 1\}$ starting with 0 and ending with 1.

Properties of Regular Expressions

Let P and Q be the regular expressions.

The properties of P and Q are as follows,

1. $L(P + Q) = L(P) \cup L(Q)$
2. $L(P \cdot Q) = L(P) \cdot L(Q)$
3. $L(L(P)) = L(P)$
4. $L(P^*) = (L(P))^*$
5. $L(P) \cdot \epsilon = \epsilon \cdot L(P) = L(P)$.

Q2. Explain about the identity rules of regular expressions.

Answer :

April-18(R16), Q5(b)

Identity Rules

Identity rules represents the equality of regular expressions with respect to specific operator and value (ϕ , ϵ).

Consider P , Q , R as regular expressions. The identity rules of regular expressions are given below. These rules help in making regular expressions more simpler.

- ❖ $\phi + R = R$
- ❖ $\phi R = R\phi = \phi$
- ❖ $\epsilon R = R\epsilon = R$
- ❖ $\phi^* = \epsilon$ and $\epsilon^* = \epsilon$

- ❖ $R + R = R$
- ❖ $RR^* = R^*R = R^+$
- ❖ $\epsilon + RR^* = R^*$ and $\epsilon + R^*R = R^*$
- ❖ $(R^*)^* = R^*$
- ❖ $R^*R^* = R^*$
- ❖ $\epsilon + R^* = R^*$
- ❖ $(R + \epsilon)^* = R^*$
- ❖ $R^*(\epsilon + R)^* = (\epsilon + R)^*R^* = R^*$
- ❖ $R^*R + R = R^*R$
- ❖ $(P + Q)R = PR + QR$ and $R(P + Q) = RP + RQ$
- ❖ $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$
- ❖ $(PQ)^*P = P(QP)^*$

Q3. Explain the terms,

- (i) Regular set
- (ii) Regular language
- (iii) Operations of regular expressions.

Answer :

(i) Regular Set

Let Σ be a finite alphabet, then a class containing sets of strings over Σ is called regular set. A regular set is recursively defined as follows,

- (a) The empty set is a regular set over Σ .
- (b) The string of length 0 is a regular set over Σ .
- (c) Each input symbol say a in Σ is a regular set over Σ .
- (d) If P and Q are two regular sets over Σ , then
 - ❖ $P \cup Q$ is a regular set over Σ
 - ❖ PQ is a regular set over Σ
 - ❖ P^* is a regular set over Σ .

Examples

- ❖ Let $\Sigma = \{0\}$, then the set of strings $\{0, 00, 000, \dots\}$ is a regular set.
- ❖ Let $\Sigma = \{0, 1\}$, then the set of strings $\{01, 10\}$ is a regular set.
- ❖ Let $\Sigma = \{a, b\}$, then the set of strings starting with a and ending with b is a regular set.

(ii) Regular Language

Regular language is a formal language generated from regular sets over input alphabet Σ .

Example

If $\Sigma = \{a, b\}$, then all set of strings that accepts even number of a 's is a regular language.

i.e., the regular language $(L) = \{aa, aab, abab, \dots\}$

Properties of Regular Language

1. A regular language is accepted by NFA, DFA and read-only by TM.
2. A regular language is described by regular expression.
3. A regular language generates a regular grammar.

Operations of Regular Expressions

(iii) Let P and Q be two regular expressions. The operations that can be performed on P and Q are as follows.

(a) Union (+)

The union (+) operator is an operator which is used to combine the elements of the two sets. It has a lower precedence when compared to other two operators (i.e., * and .).

The union of P and Q (i.e., $P + Q$ or $P \cup Q$) represents the set $\{a \cup b \mid a \in P \text{ and } b \in Q\}$.

Example

If $P = \{1, 2, 3\}$ and $Q = \{1, 2, 3, 4, 5, 6\}$ then,
 $P + Q = \{1, 2, 3\} + \{1, 2, 3, 4, 5, 6\}$
 $= \{1, 2, 3, 4, 5, 6\}$

(b) Concatenation

The concatenation or dot (.) operator is an operator which is used to concatenate the elements of two sets. It has precedence lower than star (*) operator but higher than union (+) operator.

The concatenation of P and Q (i.e., PQ or PQ) represents the set $\{ab \mid a \in P \text{ and } b \in Q\}$.

Example

If $P = \{1, 2, 3\}$ and $Q = \{1, 2\}$ then,
 $PQ = \{1, 2, 3\} \cdot \{1, 2\}$
 $= \{(1,1), (1,2), (2,1), (2,2), (3,1), (3,2)\}$

(c) Kleene Closure or Star Closure

The Kleene closure or star (*) operator is an operator which is used to obtain the set of strings including null (ϵ) from the set of alphabet (Σ). It has highest precedence than other operators.

The Kleene closure of P^* represents the smallest superset of P which holds all the strings (ϵ) developed by the concatenation of zero or more strings in P . The superset is closed under string concatenation.

Example

$\{a, bc\}^* = \{\epsilon, a, bc, abc, aa, bcabc, \dots\}$

Q4. Explain in detail about,

- (i) Building of regular expressions
- (ii) Precedence of operators of regular expressions.

Answer :

(i) Building of Regular Expressions

Building of regular expressions involves some basic expressions (such as constants, variables) and operators. A part from this, it also requires some elements (like parentheses) and methods for grouping operators with respective operands.

For example, the algebra uses constants (such as integer and real numbers), variables (such as x, a, y) and operators (like $+$, $-$, $\times$ and $/$) to build algebraic regular expressions as shown below,

$$ax + b \times 1 = 10$$

Some of the constants, variables and operators of the algebraic regular expressions are listed below,

Constants : ϵ, ϕ

Operators : Union (+), star (*), Concatenation (.).

Variables : Language L .

The following are the valid regular expressions as well as the languages represented by them.

- ❖ ϕ is a regular expression denoting the regular set $\{\phi\}$. i.e., $L(\phi) = \phi$.
- ❖ ϵ is a regular expression denoting the regular set $\{\epsilon\}$. i.e., $L(\epsilon) = \{\epsilon\}$.
- ❖ a is a regular expression denoting the regular set $\{a\}$. i.e., $L(a) = \{a\}$.

(ii) Precedence of Operators of Regular Expressions

Precedence of operators can be defined as the order in which the operators are linked to their operands. The precedence of regular-expression operators is as follows,

- (i) First priority is given to '*' operator. This means, the star operator group the operands which is immediate left to it prior to the grouping done by any other operator.
- (ii) Second priority is given to the concatenation or dot (.) operator. This means, after the * operator, the concatenation operator should be grouped with the corresponding operands. The grouping can be done from any side because, concatenation is associative. For instance, the symbols in the string 101 can be grouped either as (10)1 or 1(01). However, grouping from left is preferred.
- (iii) Third priority is given to union (+) operator. This means, grouping by this operator can be accomplished only after the grouping of star and concatenation operators. However, union is also associative and grouping from left is preferred as in concatenation operator.

Example

Consider the regular expression $10^* + 01$, the grouping done by operators is shown below,

1. The operands with * operator are grouped first as shown, $1(0^*) + 01$
2. Next, the operands with concatenation operator (i.e., 1 and (0^*) , 0 and 1) are grouped together as shown, $(1(0^*)) + (01)$
3. Lastly, the operands with union (+) operator (i.e., $(1(0^*))$ and (01)) are grouped together as shown, $((1(0^*)) + (01))$

Q5. Find regular expressions representing the following sets,

- The set of all strings over $\{0, 1\}$ having atmost one pair of 0's or atmost one pair of 1's.
- The set of all strings over $\{a, b\}$ in which the number of occurrences of a is divisible by 3.
- The set of all strings over $\{a, b\}$ in which there are atleast two occurrences of b between any two occurrences of a .
- The set of all strings over $\{a, b\}$ with three consecutive b 's.

Answer :

- (a) The Set of All Strings over $\{0, 1\}$ having atmost One Pair of 0's or Atmost One Pair of 1's

The set of all strings over $\{0, 1\}$ having atmost one pair of 0's means that the set of strings may contain no pair of 0's or may contain exactly one pair of 0's.

- Regular expression for no pair of 0's,
 $(1 + 01)^*$
- Regular expression for exactly one pair of 0's,
 $(1 + 01)^*00(1 + 01)^*$

Similarly, the set of all strings over $\{0, 1\}$ having atmost one pair of 1's means that the set of strings may contain no pair of 1's or may contain exactly one pair of 1's.

- Regular expression for no pair of 1's,
 $(0 + 10)^*$
- Regular expression for exactly one pair of 1's,
 $(0 + 10)^*11(0 + 10)^*$

So, the regular expression for the given set is obtained by combining all the above expressions.

$$(1 + 01)^* + (1 + 01)^*00(1 + 01)^* + (0 + 10)^* + (0 + 10)^*11(0 + 10)^*$$

- (b) The Set of All Strings over $\{a, b\}$ in which the Number of Occurrences of a is Divisible by 3

The number of occurrences of ' a ' is divisible by 3 means that the string may have zero a 's, three a 's, six a 's, nine a 's and so on.

- Regular expression for zero a 's,
 b^*
- Regular expression for 3 a 's, 6 a 's, 9 a 's,
 $(b^*ab^*ab^*ab^*)^*$

The required regular expression can be obtained by combining the regular expression of zero a 's, 3 a 's, 6 a 's, 9 a 's, Therefore, regular expression is,

$$b^* + (b^*ab^*ab^*ab^*)^*$$

- (c) The Set of All Strings over $\{a, b\}$ in which there are atleast Two Occurrences of b between any Two Occurrences of a

- Regular expression of the string having no a 's is b^* .
- Regular expression for only one a is b^*ab^* .
- Regular expression for atleast two occurrences of b and only one a ,
 $(b + abb)^*$
- Regular expression for atleast two occurrences of b between any two occurrences of a will be of the form,

$$(b + abb)^*ab^*$$

So, the regular expression for given set is $b^* + (b + abb)^*ab^*$.

- (d) The Set of All Strings over $\{a, b\}$ with Three Consecutive b 's

The example of strings for this language are bbb , $abbb$, $bbbab$, $ababbb$ etc. The regular expression for three consecutive b 's is of the form,

$$(S_1)bbb(S_2)$$

Where S_1, S_2 can be any number of a 's or b 's. So, the regular expression for any number of a 's or b 's is,

$$(a + b)^*$$

Therefore, the regular expression for given set is,

$$(a + b)^*bbb(a + b)^*$$

Q6. Find the regular expression corresponding to each of the following subsets over $\{0, 1\}^*$.

- The set of all strings containing no three consecutive 0's.
- The set of all strings where the 10th symbol from right end is 1.
- The set of all strings over $\{0, 1\}$ having even number of 0's and odd number of 1's.

Answer :

- (a) The Set of All Strings Containing no Three Consecutive 0's

The language denoted by the required regular expression must be of form $x001y$ where 1 after a pair of zeros ensures that there are no three consecutive 0's and x, y are strings containing any number of 1's i.e., $(1^*0011^*)^*$. In order to generate strings which ends in 0 or includes all 1's i.e., $1^*(0 + 00 + \epsilon)$ should be added.

So, the regular expression is,

$$r = (1^*0011^*)^* + 1^*(0 + 00 + \epsilon).$$

(b) The Set of All Strings where the 10th Symbol from Right End is 1

According to the required regular expression, the language will have set of strings like 0101001110111, 11000011101, etc. Here the 10th symbol from right end is 1 and other 9 symbols from the right can be 0 or 1. Similarly, the symbols from 11th position onward can be 0 or 1. So, the required regular expression is,

$$(0+1)^*1(0+1)^9$$

(c) The Set of All Strings over {0, 1} having Even Number of 0's and Odd Number of 1's

The example of strings for this language are 01110, 1000, 00011111,.....

The required regular expression can be obtained by combining the regular expression for even number of 0's and the regular expression for odd number of 1's.

Regular expression for even number of 0's is $(1^*01^*01^*)^*$.

Regular expression for odd number of 1's is $(0^*10^*10^*)^*0^*10^*$.

So, regular expression is $(1^*01^*01^*)^*(0^*10^*10^*)^*0^*10^*$.

2.1.1 Finite Automata and Regular Expressions

Q7. Prove "If L is accepted by DFA, then there exists an equivalent regular expression which develops L".

Answer :

Proof Let the language L accepted by DFA, D' be defined as $D' = \{q_0, q_1, q_2, \dots, q_n\}, \Sigma, \delta, q_0, F$

Where, $\{q_0, q_1, q_2, \dots, q_n\}$ are set of input states

Σ is set of input symbol, δ represent the transition function, q_0 is a initial state and F is a final state

In order to construct a regular expression equivalent to the given DFA, find R_{ij}^p

Where, p represents the initial states and q represents the final state.

In the path from q_i to q_j , p states are traversed,

Where, $p \leq$ The number of states in finite automata.

The following table illustrates regular expression for $p = m_i$ states

	p = 0	p = 1	p = 2	p = 3		p = n
r_{11}	r_{11}^0	r_{11}^1	r_{11}^2	r_{11}^3		r_{11}^n
r_{12}	r_{12}^0	r_{12}^1	r_{12}^2	r_{12}^3		r_{12}^n
r_{13}	r_{13}^0	r_{13}^1	r_{13}^2	r_{13}^3		r_{13}^n
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
r_{1n}	r_{1n}^0	r_{1n}^1	r_{1n}^2	r_{1n}^3		r_{1n}^n
r_{21}	r_{21}^0	r_{21}^1	r_{21}^2	r_{21}^3		r_{21}^n
r_{22}	r_{22}^0	r_{22}^1	r_{22}^2	r_{22}^3		r_{22}^n
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
r_{2n}	r_{2n}^0	r_{2n}^1	r_{2n}^2	r_{2n}^3		r_{2n}^n
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
r_{n1}	r_{n1}^0	r_{n1}^1	r_{n1}^2	r_{n1}^3		r_{n1}^n
r_{n2}	r_{n2}^0	r_{n2}^1	r_{n2}^2	r_{n2}^3		r_{n2}^n
r_{nj}	r_{nj}^0	r_{nj}^1	r_{nj}^2	r_{nj}^3		r_{nj}^n
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
r_{nn}	r_{nn}^0	r_{nn}^1	r_{nn}^2	r_{nn}^3		r_{nn}^n

Table: Regular Expression for $p = m_i$

In the above table,

Number of rows = n^2 and

Number of columns = $n + 1$

The table involves multiple columns. Hence, subscripts notation are used. For instance,

r_{ij} represents regular expression with i, j subscripts.

It is to be noted that the regular expression $r_{11}^1, r_{11}^2, r_{11}^3, \dots, r_{11}^{n-1}$ are different from one another.

The entries in the table are constructed as follows,

For $p = 0$

Case(I)

If $i \neq j$ then

$$r_{ij} = \{\text{Any input symbol } (a) / \delta(q_i, a) = q_j\}$$

Case(II)

If $i = j$ then

$$r_{ii}^0 = \{\text{Any input symbol } (a) / \delta(q_i, a) = q_i\} \cup \{\epsilon\}$$

r_{ii}^0 can be any of the following in the set $\{\phi, \epsilon, a, a + \epsilon\}$

It is important to note that Kleen closure (*) or concatenation is not involved in $k = 0$ column.

For $p = 1$ column,

$$r_{ij}^1 = r_{ii}^0 (r_{ij}^0)^* r_{jj}^0 + r_{ij}^0$$

For $p = 2$,

$$r_{ij}^2 = r_{ii}^1 (r_{ij}^1)^* r_{jj}^1 + r_{ij}^1$$

$\vdots$

The general formula for $p = p$ column is,

$$r_{ij}^p = r_{ii}^{p-1} (r_{ij}^{p-1})^* r_{jj}^{p-1} + r_{ij}^{p-1}$$

Equivalence of NFA and Regular Expression

Proof

By applying induction method, the following theorem can be proved.

The basic step of induction is to consider that r has zero operators.

Since r contains zero operator it can take values either ϕ or 0 for some 0 in input set Σ .

The finite automata for $r = \epsilon, \phi, 0$ is shown the figure (1),

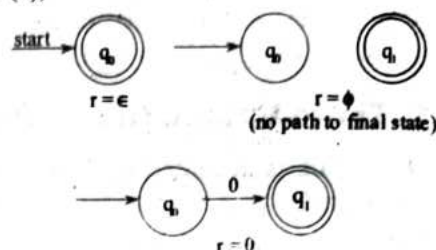


Figure (1): Finite Automata for Given Regular Expression

Introduction

To prove that the theorem is true for n number of operators, consider $n \geq 1$ i.e., the regular expression may contain either one or more than one operators.

In any regular expression there exist three possible cases.

1. Union
2. Concatenation
3. Closure.

Case-1: Union Case

Let r_1 and r_2 be the two regular expressions such that $r = r_1 + r_2$.

Consider that,

There exists two NFA's $M_1 = (Q_1, \Sigma_1, \delta_1, \{f_1\})$ and $M_2 = (Q_2, \Sigma_2, \delta_2, \{f_2\})$ in such a way that

$$L(M_1) = L(r_1).$$

That is the language defined by regular expression r_1 is equal to that of M_1 . Similarly $L(M_2) = L(r_2)$.

Let Q_1 be the set of finite states of M_1 .

Let Q_2 be the set of finite states of M_2 .

Assume that Q_1 and Q_2 are completely different from each other i.e., Q_1 and Q_2 are disjoint.

Let q_0 and f_0 be new initial and final state respectively such that,

$$M = ((Q_1 \cup Q_2 \cup \{q_0, f_0\}), (\Sigma_1 \cup \Sigma_2), \delta, q_0, \{f_0\})$$

The transition function δ is given by,

- (i) $\delta(q_0, \epsilon) = \{q_1, q_2\}$
- (ii) $\delta(q, 0) = \delta_1(q, 0)$ for q in $Q_1 - \{f_1\}$ and 0 in $\Sigma_1 \cup \{\epsilon\}$
- (iii) $\delta(q, 0) = \delta_2(q, 0)$ for q in $Q_2 - \{f_2\}$ and a in $\Sigma_2 \cup \{\epsilon\}$
- (iv) $\delta(f_1, \epsilon) = \delta_1(f_1, \epsilon) = \{f_0\}$

Q8. Discuss the method for conversion of regular expression to finite automata.

Answer :

Conversion of Regular Expression to Finite Automata

Regular expressions and finite automata are said to be equivalent, because the languages which are generated by regular expressions are also accepted by finite automata and vice-versa.

The equivalence of finite automata and regular expressions can be proved by the following two consideration.

1. Initially it must be proved that for a given regular expression (r) there exist a corresponding NFA with ϵ -transition accepting the same language, i.e., $L(A) = L(R)$.
2. Finally, it must be proved for a given finite automata there exist a corresponding regular expression generating the same language, i.e., $L(R) = L(A)$.

The moves that are present on machine M is shown in the figure (2).

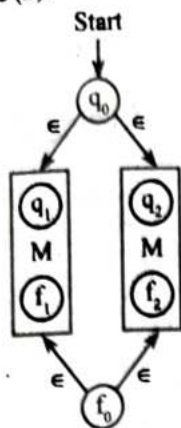


Figure (2): Union Operation

In the figure (2), the transition from q_0 to f_0 starts from q_1 or q_2 on input ϵ . If the path goes to q_1 , then it follows the path in machine M_1 and goes to the state f_1 and then to f_0 on ϵ . Similarly, if the path goes to q_2 , then it follows the path in machine M_2 and goes to state f_2 and then to f_0 on ϵ . Thus the $L(M) = L(M_1) \cup L(M_2)$. That means either the path in machine M_1 or M_2 will be followed.

Case-2: Concatenation Case

Let, r_1 and r_2 are two regular expressions. Such that $r = r_1 r_2$. Let the M_1 and M_2 be machines where in $L(M_1) = L(r_1)$ and $L(M_2) = L(r_2)$.

The machine M will be,

$$M = (Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, \delta, \{q_1\}, \{f_2\})$$

δ is given as,

- $\delta(q, 0) = \delta_1(q, 0)$ for q in $Q_1 - \{f_1\}$ and α in $\Sigma_1 \cup \{\epsilon\}$
- $\delta(f_1, \epsilon) = \{q_2\}$
- $\delta(q, 0) = \delta_2(q, 0)$ for q in Q_2 and α in $\Sigma_2 \cup \{\epsilon\}$.

The transition of M is as follows,



Figure (3): Machine M for Concatenation

In the figure (3), the initial state q_1 on input 0 goes to next state i.e., f_1 on receiving ϵ the transition maps from f_1 to q_2 . On receiving input 1 the transition gives from q_2 to f_2 .

Thus, $L(M) = 01$

i.e., 0 is in $L(M_1)$ and 1 is in $L(M_2)$.

$$L(M) = L(M_1) L(M_2).$$

Case-3: Closure Case

Let r_1 be regular expression such that $r = r_1^*$

Let $r = r_1^*$ where r_1 be a regular expression.

M_1 will be such that $L(M_1) = L(r_1)$.

Then construct $M = (Q_1 \cup \{q_0, f_0\}, \Sigma_1, \delta, q_0, \{f_0\})$

δ is given by,

- $\delta(q_0, \epsilon) = \delta(f_1, \epsilon) = (q_1, f_0)$
- $\delta(q, 0) = \delta_1(q, 0)$ for q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\epsilon\}$

The machine M is given as,



Figure (4): Closure Case

In the figure (4) the machine M_1 start from q_0 on receiving input ϵ it goes to q_1 , there is a path. The exists a path from f_1 to q_1 . Similarly there is a transition from f_1 to f_0 final state on input ϵ .

Thus, $L(M) = L(M_1)^*$ is proved.

Equivalence of DFA and Regular Expression

Let the DFA $M = (\{q_0, q_1, \dots, q_n\}, \Sigma, \delta, q_0, F)$ accepts a set of language L . The above finite automata accepts set of strings $\omega \in \Sigma^*$ if there exist a path from initial state q_0 to some final state q_f . This path consists of subsets of Σ^* from q_0 to q_f which helps in representing the subsets of Σ^* by regular expression.

The basic aim of algorithm is to find the subsets of path values from q_i to q_j .

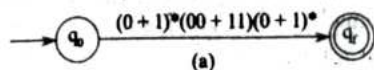
Q9. Construct a DFA for the regular expression $(0 + 1)^*(00 + 11)(0 + 1)^*$.

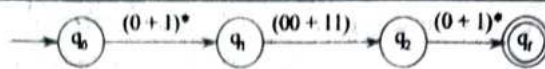
Answer : (Dec.-18(R16), Q4(b) | April-18(R16), Q5(a))

Given regular expressions,

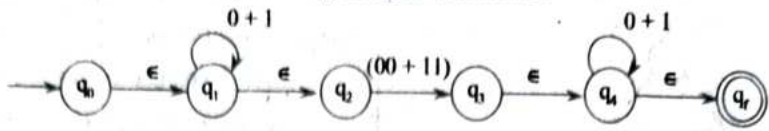
$$r = (0 + 1)^* (00 + 11) (0 + 1)^*$$

Step1: Construct a transition graph equivalent to given regular expression using NFA with ϵ -moves.

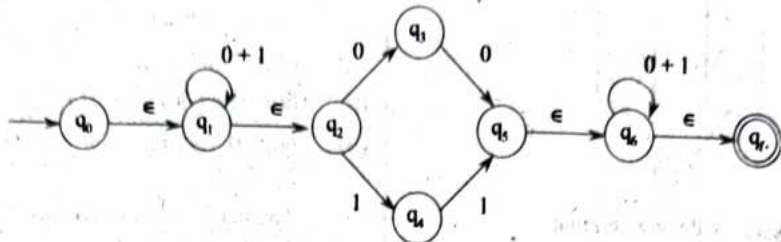




(b) Eliminate concatenation

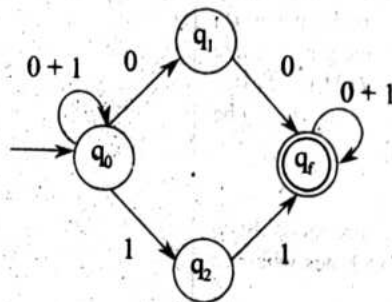


(c) Eliminate * operation



(d) Eliminate concatenation and +

Step2: Convert NFA with ϵ -moves to NFA without ϵ -moves.

Figure: NFA for $r = (0 + 1)^*(00 + 11)(0 + 1)^*$

Step3: Transition table for NFA is,

q / Σ	0	1
$\rightarrow q_0$	$[q_0, q_1]$	$[q_0, q_2]$
q_1	q_f	ϕ
q_2	ϕ	q_f
q_f	q_f	q_f

The transition table for equivalent DFA is,

q / Σ	0	1
$\rightarrow q_0$	$[q_0, q_1]$	$[q_0, q_2]$
$[q_0, q_1]$	$[q_0, q_1, q_f]$	$[q_0, q_2]$
$[q_0, q_2]$	$[q_0, q_1]$	$[q_0, q_2, q_f]$
$[q_0, q_1, q_f]$	$[q_0, q_1, q_f]$	$[q_0, q_2, q_f]$
$[q_0, q_2, q_f]$	$[q_0, q_1, q_f]$	$[q_0, q_2, q_f]$

From the above table it can be inferred that, $[q_0, q_1, q_f]$ and $[q_0, q_2, q_f]$ have identical transitions on inputs 0 and 1. So, the number of states of DFA are reduced and all the occurrences of $[q_0, q_2, q_f]$ are replaced by $[q_0, q_1, q_f]$.

The DFA with reduced number of states is shown in figure.

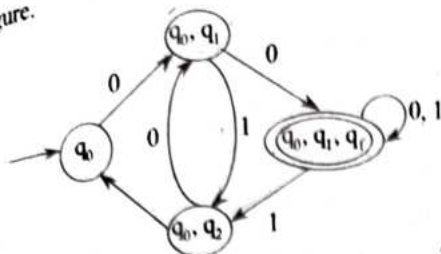


Figure: DFA with Reduced Number of States

Q10. Design a FA for the following languages.

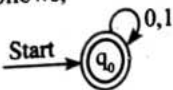
- (a) $(0^*1^*)^*$ (b) $(0+1)^*111^*$
(c) (0^*11^*+101) .

Answer : (Dec.-18(R16), Q5 | May-17(R13), Q5)

(a) $(0^*1^*)^*$

Given regular expression is $RE = (0^*1^*)^*$

The finite automata for given regular expression is constructed as follows,

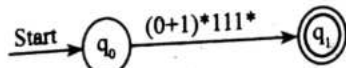


(b) $(0+1)^*111^*$

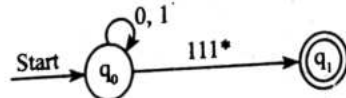
Given regular expression is, $RE = (0+1)^*111^*$

The finite automata for the given regular expression is constructed as follows,

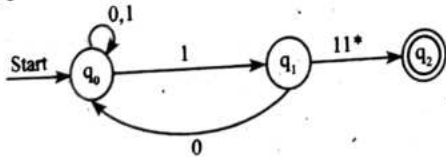
Step 1



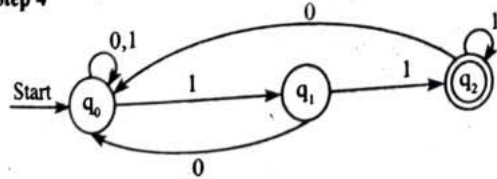
Step 2



Step 3



Step 4

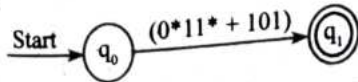


(c) (0^*11^*+101)

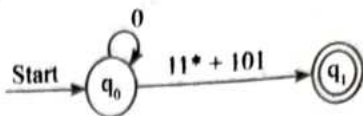
Given regular expression is $RE = (0^*11^*+101)$

The finite automata for regular expression is constructed as follows,

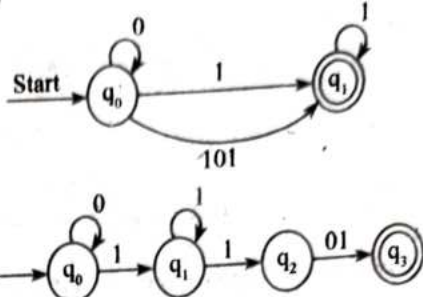
Step 1



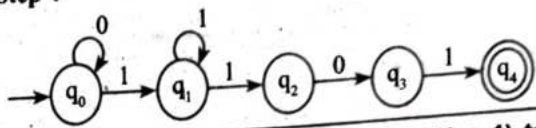
Step 2



Step 3



Step 4



Q11. Convert regular expression (01^*+1) to finite automata.

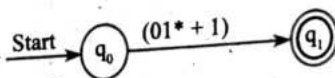
Dec.-19(R16), Q4

Answer :

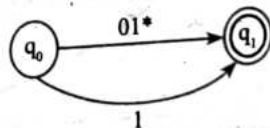
The given regular expression is, $RE = (01^*+1)$

The FA for regular expression is,

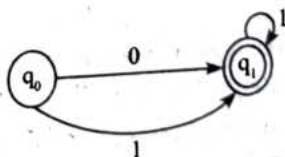
Step 1



Step 2



Step 3



Q12. Convert the following regular expressions to NFA with epsilon transitions,

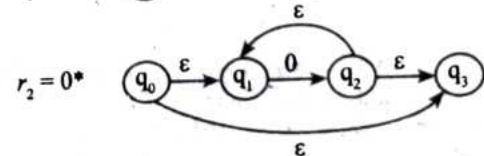
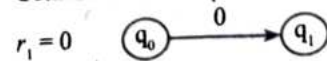
- (a) 0^*+1101 (b) $(0+1)^*$.

Answer :

May-16(R13), Q3

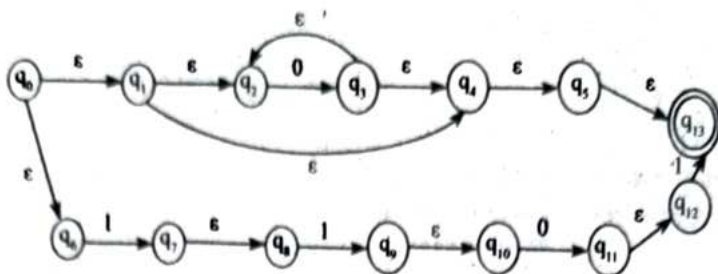
(a) Given regular expression is (0^*+1101)

Consider NFA for $r_1 = 0$



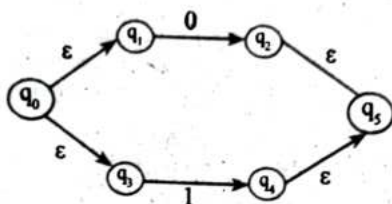
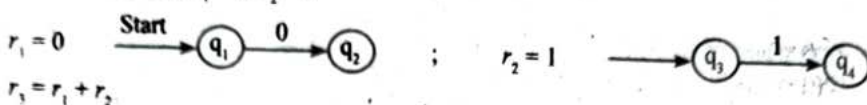


NFA for $0^* + 1101$ is,

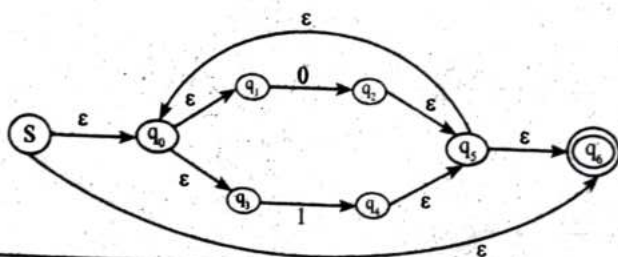


(b) Given regular expression is $(0 + 1)^*$

Consider NFA for $r_1 = 0, r_2 = 1$



$r_4 = r_3^*$



Q13. Convert the regular expression $((00)^*(11) + 01)^*$ into an NFA.

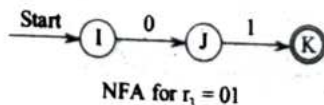
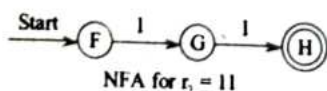
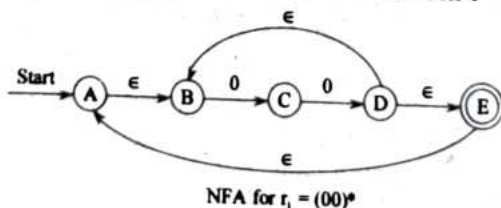
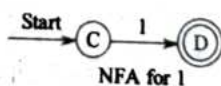
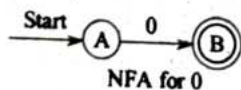
Answer :

The given regular expression R can be written as,

$$(((00)^*(11)) + 01)^*$$

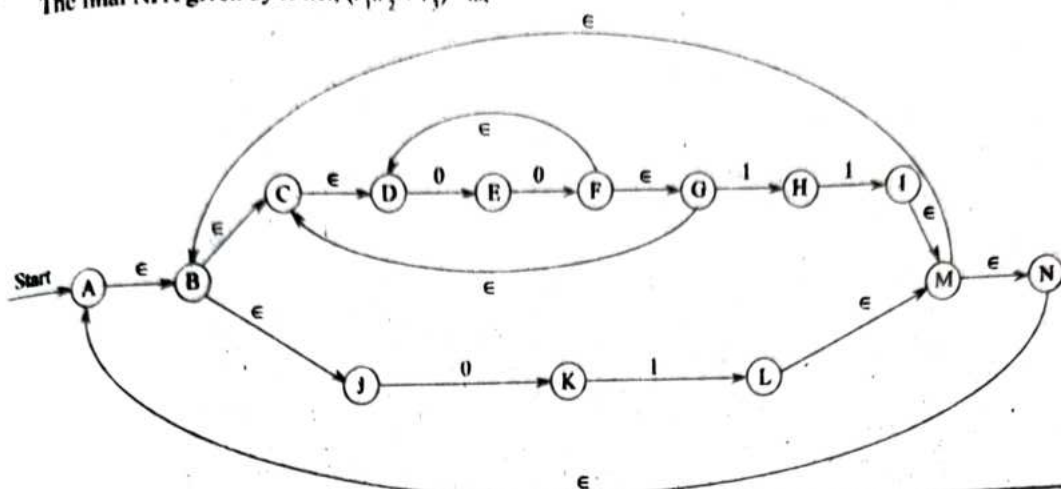
$$\text{Let, } R = (r_1 + r_2 + r_3)^*$$

Where, $r_1 = (00)^*, r_2 = 11, r_3 = 01$



Jan./Feb.-23(R18), Q2(b)

The final NFA given by R i.e., $(r_1 r_2 + r_3)^*$ is,



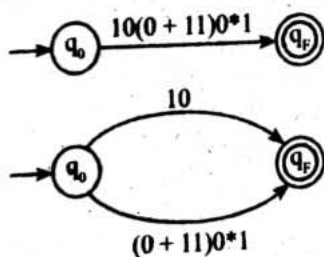
Q14. Construct a NFA equivalent to the regular expression $10(0+11)0^*1$.

Answer : Jan./Feb.-23(R18), Q4(b)

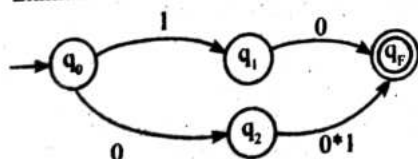
Given regular expression is,
 $r = 10(0 + 11)0^*1$.

Step 1

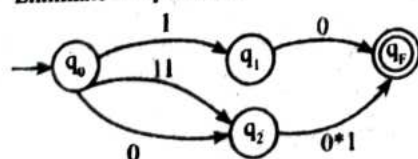
(a) Construct a transition graph equivalent to r using NFA with ϵ -moves,



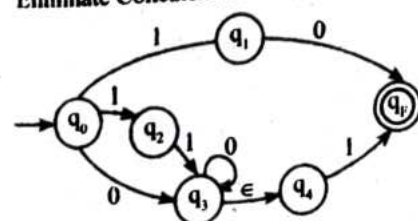
(b) Eliminate Concatenation



(c) Eliminate + Operations



(d) Eliminate Concatenation and *



Step 2

Convert NFA with ϵ -moves to NFA without ϵ -moves.

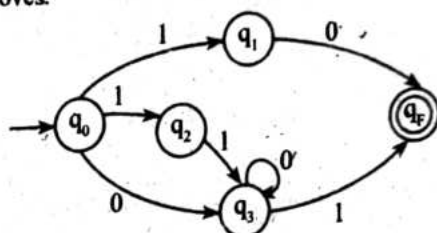


Figure: NFA for $10(0 + 11)0^*1$

Transition table for NFA is,

q/Σ	0	1
q_0	q_3	$\{q_1, q_2\}$
q_1	q_f	ϕ
q_2	ϕ	q_3
q_3	q_3	q_f
q_f	ϕ	ϕ

NFA for $R = (r_1 r_2 + r_3)^* = ((00)^* 11) + 01)^*$

Q15. Construct the Finite Automaton to accept the regular expression $1^*01(0+11)^*$.

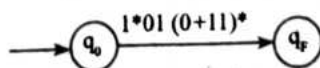
Answer :

Feb.-22(R18), Q3(a)

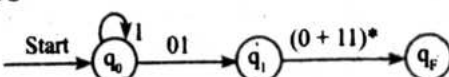
Given regular expression is,
 $R.E = 1^*01(0 + 11)^*$

The transition diagram for the given regular expression is constructed in the following manner,

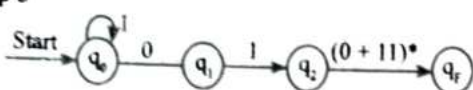
Step 1



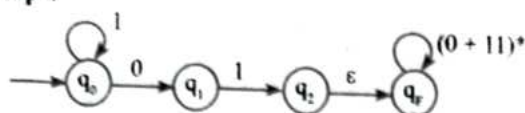
Step 2



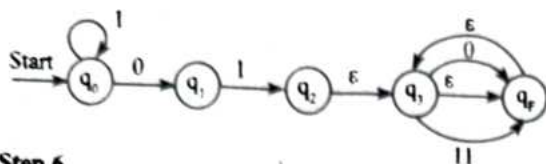
Step 3



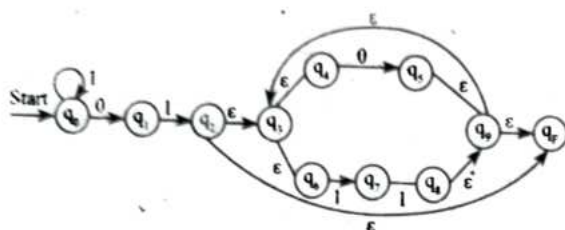
Step 4



Step 5

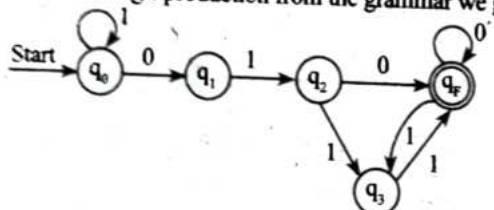


Step 6



NFA to accept $1^*01(0+11)^*$

Removing ϵ production from the grammar we get,



Finite Automata for expression $1^*01(0+11)^*$

The transition table for above finite automata is,

State	Input symbol	
	0	1
q_0	q_1	q_0
q_1	ϕ	q_2
q_2	q_f	q_3
q_3	ϕ	q_f
q_f	q_f	q_3

Since all the input states in the above transition table reaches to a unique state. The above constructed finite automata is deterministic.

2.1.2 Applications of Regular Expressions, Algebraic Laws of Regular Expressions

Q16. Discuss the applications of a regular expressions.

Answer :

Applications of Regular Expressions

Regular expressions have a wide variety of applications ranging from compilers to search engines. These are widely used in lexical analysis and software engineering applications.

1. Regular expressions are used in web search engines to search the keywords through a database of file names on public FTP servers.
2. They are most useful in smaller search engines like find/replace tool in text editor. They are also used in UNIX tools like grep.
3. Regular expressions Play a major role in software engineering. To ensure reusability, component based software applications are designed now-a-days. The problem with such applications now-a-days while integrating them if interaction between them is not properly defined.

The performance can be unpredictable. Regular expressions are used for defining proper specification so that it is made easier to understand. They are also used in characterization of test cases. Regular language theory is used in handling paths and it makes coverage analysis and test case generation simple. Regular expressions also have their role in testing. At integration level, a relation is established between finite automata and regular languages.

4. The most important area where regular expressions are used is lexical analysis. It is a process of dividing input program into a sequence of tokens. Lexical analysis involves two major functions i.e., scanner and tokenizer. The best way of defining a token is by regular expression. Scanning is a process of using which a set of regular expressions can be defined, each matching a set of valid lexemes that belong to this type.

Tokenizer uses finite state machine to define lexical grammar of the language being analyzed which in turn needs regular expressions.

Q17. Write in brief about the algebraic rules for regular expressions.

Answer :

Jan./Feb.-23(R18), Q5(a)

There are some set of identity rules that can be used to express a regular expression in a simplified form. But these rules are applicable only if the expression is in required form. Hence to obtain the required form of expression it must be rewritten without having any impact on the language. The means of modifying an expression can be determined by applying the following set of algebraic laws.

1. Commutative Law

This law states that if an operator interchanges the order of its operand then the result must remain the same.

Example: Commutative law of addition is,

$$A + B = B + A$$

A regular expression supports the commutative law for union. It states that union of any two languages can be determined in either order. Mathematically it can be represented as, $L + M = M + L$

2. Associative Law

This law states that an operator may have the authority to regroup its operands when more than one operands are used in an expression.

Example: Associative law of addition is,

$$(A + B) + C = A + (B + C)$$

A regular expression supports the associative law for union and concatenation.

(i) The associative law for union states that union of any three languages can be obtained in either of the way.

(a) Starting the union operation with first two operand and ending with last operand.

(b) Starting the union operation with last two operand and ending with first operand.

The mathematical representation of associative law for union is, $(L + M) + N = L + (M + N)$

(ii) The associative law for concatenation of three language is, $(LM)N = L(MN)$

3. Identity Law

This law states that when a value is assigned to an operator along with its identity then the result will be the value itself.

Example: The additive identity of A with 0 is A .

$$\text{i.e., } A + 0 = 0 + A = A$$

A regular expression support two law of identity they are as follows,

(i) Identity law for union $= \phi + L = L + \phi = L$

(ii) Identity law for concatenation $= \epsilon L = L\epsilon = L$

4. Annihilators Law

This law states that when a value is assigned to an operator along with a annihilator then the result will be the annihilators.

Example: The additive annihilator of A is 0

$$A \times 0 = 0 \times A = 0$$

A regular expression support annihilator for concatenation i.e., $\phi L = L\phi = \phi$

5. Distributive Law

This law is applicable on an expression containing two operators.

Example: Distributive law of multiplication over addition operator is, $A \times (B + C) = (A \times B) + (A \times C)$

A regular expression supports distributive law for concatenation over union. Since concatenation is commutative.

6. Idempotent Law

This law states that when an operator is assigned between two similar operands, the result obtained will be the same operand. Generally a common arithmetic operator does not support idempotent law.

$$\text{i.e., } A + A \neq A \quad \text{or} \quad A \times A \neq A$$

However, for some value of A this law is valid i.e.,

$$0 + 0 = 0 \quad \text{or} \quad 0 \times 0 = 0$$

A regular expression supports laws for union and intersection. The idempotent law of union is $L + L = L$.

2.1.3 Conversion of Finite Automata to Regular Expressions

Q18. Write the steps to construct regular expression from given DFA.

Answer :

Jan./Feb.-23, (R18), Q4(a)

The procedure for converting finite automata to regular expression is as follows,

1. If there are $q_1, \dots, q_f$ states in the finite automata then consider q_1 as initial state and q_f as final state and also consider α_{q_1} as transaction from q_1 to q_1 .

2. Compute the initial state q_1 using the formula

$$q_1 = \alpha_{q_1} q_1 + \epsilon$$

Initial state = Source of input state. Transaction on some input alphabet coming to it $+\epsilon$.

Similarly compute all the other state using formula.

$$q_i = \alpha_{q_i} q_i$$

3. Apply Arden's theorem i.e., $R = Q + RP$ reduced to $R = QP^*$ on the final state. This state gives the regular expression.

Q19. Explain how DFA's are converted to regular expressions by eliminating states.

Answer :

Conversion of DFA's to Regular Expressions by Eliminating States

The conversion of DFA's to Regular Expressions (RE) by state elimination method involves the following steps,

Step1: Eliminate the intermediate step first then go for accepting states. Name the new path after every elimination with the help of earlier named paths (path names are regular expressions). Repeat elimination until all the accepting states has been eliminated and left with the only start (S) and final (F) states.

Step2: Check whether the remaining (S&F) states are equal or not. The two states are said to be equal if both the states are either accepting or non-accepting. In case, if they are not equal then the automaton will have two states.

Step3: Check whether the start state is an accepting state. If so, then eliminate all the states except the start state. The resulting automaton is now one state automaton.

Step4: Finally, join all the regular expressions obtained at every elimination in the above steps.

The above steps are explained clearly through the following sample automaton.

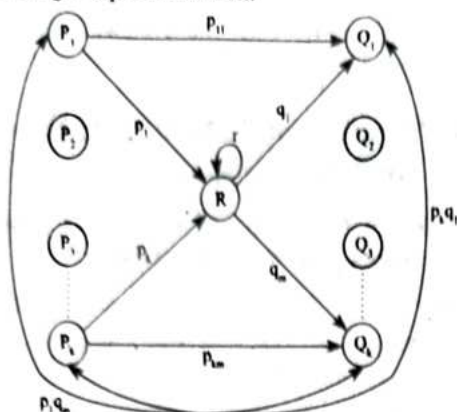


Figure: Sample Automaton

$P_1, P_2, \dots, P_k, R, Q_1, Q_2, \dots, Q_m$ are the states of automaton. P_{11} is the path name from state P_1 to Q_1 . Similarly, $P_1, q_1, P_k, q_m, P_{km}, P_{1m}, P_k q_1$ are path names for the corresponding states. r is the self loop of state R .

Consider state R has to be eliminated. This requires elimination all the paths involving state R . Thus the resultant automaton will be,

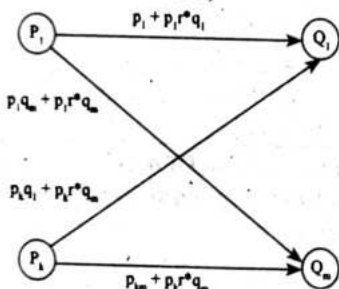


Figure: Automaton After Eliminating State R

Incase, if the states are not equal the automaton will be as shown below,

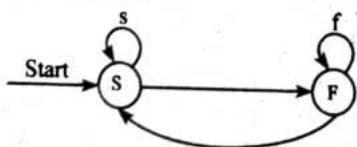


Figure: Automaton with different States

Thus, the regular expression for the above automaton will be,

$$(r + af^*b)$$

Incase, if the start state is also an accepting states, then the state F is eliminated and resulting automaton will be,

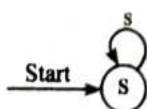


Figure: Automaton with Start State as an Accepting State

Thus, the regular expression for the above automaton will be S^* .

Q20. State and prove Arden's theorem.

Answer :

Arden's Theorem

Arden's theorem states that if P and Q are two regular expressions on input string ' Σ ' then the regular expression from $R = Q + RP$ gives unique solution as

$$R = QP^*$$

only if P does not contain a null string ϵ .

Proof

Arden's theorem states that $R = Q + RP$ reduces to $R = QP^*$

Consider the following equation,

$$R = Q + RP$$

As it is known that $R = QP^*$

Substitute $R = QP^*$ in RHS of equation (1). Then the following equation is derived,

$$\begin{aligned} R &= Q + RP \\ &= Q + QP^*P \\ &= Q(\epsilon + P^*P) \\ &= QP^* \quad [\because \epsilon + R^*R = R^*] \end{aligned}$$

Inorder to prove that the regular expression

$R = QP^*$ has unique solution

Substitute $R = Q + RP$ in RHS and LHS of equation (1).

$$\begin{aligned} R &= Q + RP \\ Q + RP &= Q + (Q + RP)P \\ &= Q + QP + RP^2 \end{aligned}$$

Similarly, replace R of equation (2) with $Q + RP$. Hence the above equation becomes,

$$\begin{aligned} &= Q + QP + (Q + RP)P^2 \\ &= Q + QP + QP^2 + RP^3 \end{aligned}$$

On further substitution the above equation will be of form.

$$\begin{aligned} Q + RP &= Q + QP + QP^2 + QP^3 + \dots QP^i + RP^{i+1} \\ &= Q(\epsilon + P + P^2 + P^3 + \dots P^i) + RP^{i+1} \end{aligned}$$

$$\therefore R = Q(\epsilon + P + P^2 + P^3 + \dots P^i) + RP^{i+1}$$

for $i \geq 0$

The above equation is of form,

$$R = QP^* + RP^{i+1}$$

Where, $P^* = (\epsilon + P + P^2 + \dots P^i)$

And P^{i+1} contains strings not less than $i+1$. Since W does not contain set RP^{i+1} , the equation becomes,

$$R = QP^*$$

Hence, it is proved that $R = QP^*$ has unique solution.

Arden's theorem is mainly used to check whether the two regular expression is equivalent or not and also for converting finite automata to regular expression.

Q21. Discuss the basic approach to convert from NFA to Regular expression. Illustrate with an example.

Answer :

The basic approach to convert NFA to Regular expression is given below,

The NFA $M = (Q, \Sigma, \delta, q_0, F)$ can be converted to a regular expression by applying State Elimination Technique.

The procedure is as follows,

Step1: Insert one new initial state and one final state by making initial as non-initial and final as non-final state.

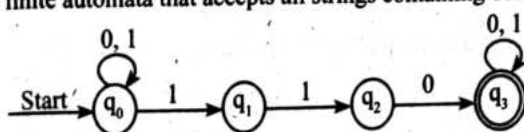
Step2: Add ϵ -transitions to the initial state and final state.

Step3: Frequently, eliminate the states one-by-one, from initial to final state except the initial and final state.

Step4: If there exists only a single transition from initial to final state, then the final regular expression can be obtained.

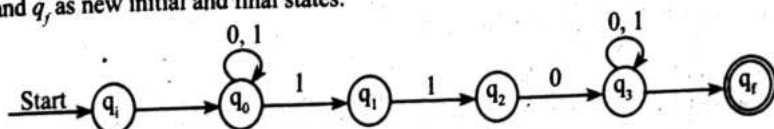
Example

The non-deterministic finite automata that accepts all strings containing 110 is as follows,

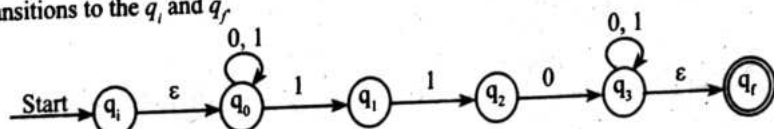


The above NFA can be converted to regular expression in the following manner,

Step1: Insert q_i and q_f as new initial and final states.

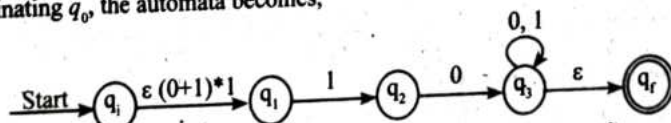


Step2: Add ϵ -transitions to the q_i and q_f .

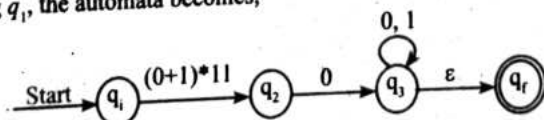


Step3: The State Elimination Technique is applied in following manner,

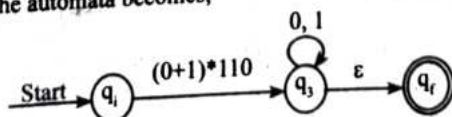
(i) On eliminating q_0 , the automata becomes,



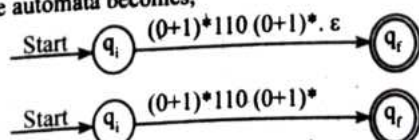
(ii) On eliminating q_1 , the automata becomes,



(iii) On eliminating q_2 , the automata becomes,

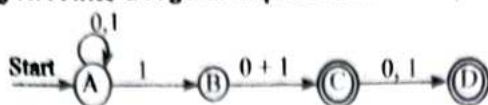


(iv) On eliminating q_3 , the automata becomes,



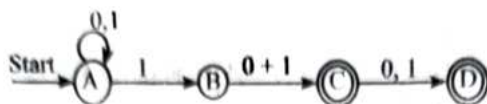
Step4: Since there is only one transition from q_i to q_f , thus the regular expression is, $RE = (0 + 1)^* 110 (0 + 1)^*$

Q22. Convert the following NFA into a regular expression.



Answer :

Given NFA is,



Since, the above NFA has two final states (i.e., C and D), the regular expression for it can be obtained based on the following considerations.

Step A

Initially, obtain the regular expression from start state to last final state (i.e., from A to D) by applying state elimination technique.

Step B

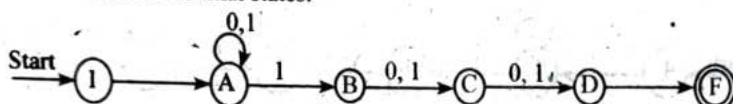
Later, eliminate the last final state (i.e., D) and then obtain regular expression from initial to first final state (i.e., from A to C).

Step C

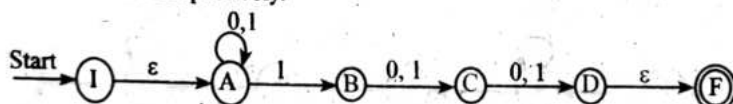
Finally combine the regular expressions of the step A and step B to obtain final regular expression.

Now, let us obtain the regular expression by considering the final state 'D' of the above NFA.

Step1: Insert I and F as new initial and final states.

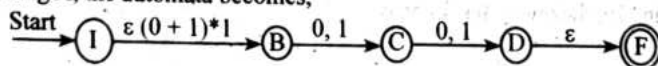


Step2: Add ϵ -transitions to I and F respectively.

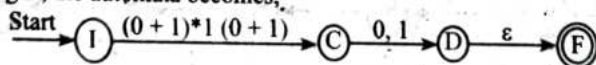


Step3: State elimination can be done in the following manner,

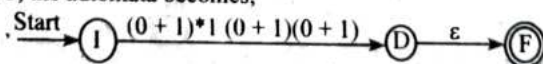
(i) On eliminating A, the automata becomes,



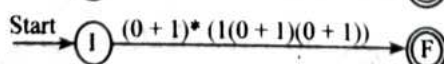
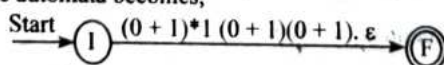
(ii) On eliminating B, the automata becomes,



(iii) On eliminating C, the automata becomes,



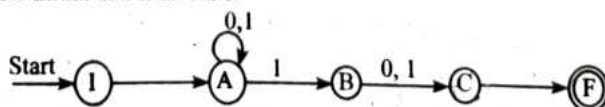
(iv) On eliminating D, the automata becomes,



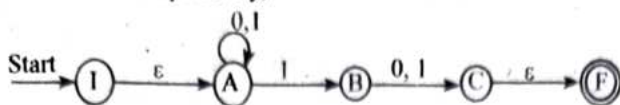
Step4: Since, there is only one transition from I to F, thus the regular expression is $RE_1 = (0+1)^* (1(0+1)(0+1))$.

Now obtain the regular expression by considering the final state 'C' as follows,

Step1: Insert I and F as new initial and final state.

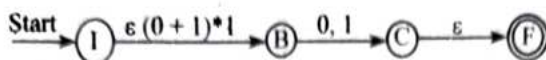


Step2: Add ϵ -transitions to the I and F respectively,

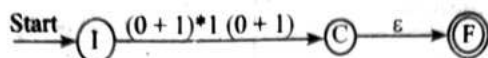


Step3: State elimination can be done in the following manner,

(i) On eliminating A, the automata becomes,



(ii) On eliminating B, the automata becomes,



(iii) On eliminating C, the automata becomes,



Step4: Since, there is only one transition from I to F, thus the regular expression is,

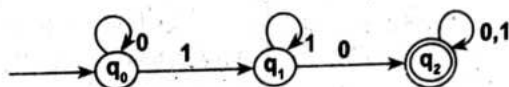
$$RE_2 = (0 + 1)^* 1(0 + 1)$$

Therefore, the final regular expression is as follows,

$$RE = RE_1 + RE_2$$

$$RE = (0 + 1)^* (1(0 + 1) (0 + 1)) + (0 + 1)^* 1(0 + 1).$$

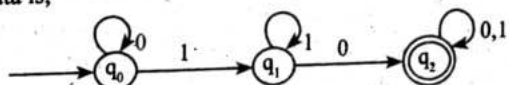
Q23. Construct the regular expression corresponding to the language accepted by following DFA.



May-19(R16), Q5

Answer :

The given finite automata is,



The regular expression for the given finite automata will be,

$$q_0 = q_0 0 + \epsilon \quad \dots (1)$$

$$q_1 = q_0 1 + q_1 1 \quad \dots (2)$$

$$q_2 = q_1 0 + q_2 (0 + 1) \quad \dots (3)$$

Since q_2 is the final state. Consider, equation (3),

$$q_2 = q_1 0 + q_2 (0 + 1)$$

The above equation is of form,

$$R = Q + RP$$

Where, $R = q_2$, $Q = q_1 0$, $P = (0 + 1)$, Thus $R = Q + RP \Rightarrow QP^*$

$$q_2 \Rightarrow q_1 0 (0 + 1)^* \quad \dots (4)$$

Now, consider equation (1),

$$q_0 = \epsilon + q_0 0$$

The above equation is of form,

$$R = Q + RP$$

Here, $R = q_0$, $Q = \epsilon$, $P = 0$

Thus,

$$q_0 = \epsilon.0^* \quad [\because R = Q + RP \Rightarrow QP^*]$$

$$\Rightarrow q_0 = 0^* \quad [\because \epsilon.R = R]$$

Substituting $q_0 = 0^*$ in equation (2), the following is obtained.

$$q_1 = 0^* 1 + q_1 1$$

The above equation is of form $R = Q + RP$

Here,

$$R = q_1, Q = 0^* 1, P = 1$$

Thus,

$$q_1 = 0^* 1 (1^*) \quad [\because R = Q + RP \Rightarrow QP^*]$$

Substitute $q_1 = 0^* 1 (1^*)$ in equation (4).

$$\begin{aligned} q_2 &= q_1 0 (0 + 1)^* \\ &= 0^* 1 (1^*) 0 (0 + 1)^* \end{aligned}$$

$\therefore$ The regular expression R.E is,

$$RE = 0^* 1 (1^*) 0 (0 + 1)^*$$

2.2 PUMPING LEMMA FOR REGULAR LANGUAGES: STATEMENT OF THE PUMPING LEMMA, APPLICATIONS OF THE PUMPING LEMMA

Q24. Explain about pumping lemma.

Dec.-19(R16), Q5(b)

OR

Discuss in detail the pumping lemma for regular languages.

Answer :

Pumping Lemma for Regular Languages

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite state machine with n states. Let L be a regular language accepted by M . Then a string $Z \in L$ such that $|Z| \geq n$ can be written as $Z = uvw$ satisfying following conditions,

- ❖ $v \neq \epsilon$
- ❖ $|uvw| \leq n$ and
- ❖ The string $uv^i w \in L$ for all $i \geq 0$.

The theorem states that given a string Z belonging to L always find a substring V near the beginning of Z that can be pumped or repeated as many times as users like or deleting it, places the resulting string in the language.

Proof

Suppose L is a regular language accepted by DFA, $M = (Q, \Sigma, \delta, q_0, F)$ with n states. Consider an input string $Z = a_1 a_2 a_3 \dots a_m$ where $m \geq n$ and a_i is an input symbol in Σ .

Pumping Lemma is used to prove that certain languages are not regular. It is useful to determine whether the language accepted by a finite automata is finite or infinite.

Now, define state p_i for $i = 0, 1, \dots, n$

Where, δ is a transition function and $q_0 = p_0$ is the initial state.

As $p_i = \delta(p_0, a_1 a_2 \dots a_i)$

So, the string Z can be written as,

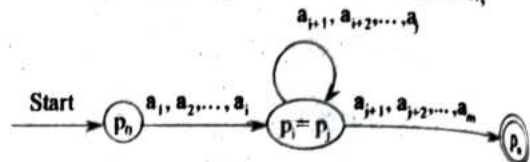
$$Z = uvw$$

Where, $u = a_1 a_2 \dots a_i$

$$v = a_{i+1} a_{i+2} \dots a_j$$

$$w = a_{j+1} a_{j+2} \dots a_m$$

This can be represented as shown below,



Figure

It is observed that if $i = 0$ then, u may be empty.

If $j = n = m$, w may be empty but v cannot be empty since $i < j$ strictly.

Consider the automation on receiving of input $uv^i w$ for $i \geq 0$.

Case 1: $i = 0$

In this case, FA goes from initial state p_0 to p_i on input u . Since $p_i = p_n$, it goes to accepting state p_n from p_i on input w . Hence it accepts string uw .

Case 2: $i > 0$

In this case, FA goes from initial state p_0 to p_i on input u , repeat ' v ' times to p_i itself on input v , and then goes to p_n on input w . Hence it accepts $uv^i w$ for all $i \geq 0$.

That is $uv^i w$ is in L .

Hence theorem is proved.

Example

Given $L = \{a^{2n} | n \geq 1\}$ consists of set of strings of the length of $2n$ i.e., strings of even length.

Assume L is regular and n be the number of states in the FA.

$$L = \{aa, aaaa, aaaaaa, \dots\}$$

Select a string Z such that $|uv| \leq n$ and $|v| > 0$.

Let,

$$Z = uvw = aaaa = aa^2 a$$

Where,

$$u = a, v = a^2 \text{ and } w = a$$

By Pumping Lemma,

$$uv^2 w = a(a^2)^2 a$$

$$= aaaaaa$$

$$|uv^2 w| = |aaaaaa|$$

$$= 6 \text{ which is even.}$$

i.e., $uv^2 w \in L$

Hence, L is regular.

Q25. Discuss in brief about applications of pumping lemma.

Answer : Jan./Feb.-23(R18), Q5(b)

Applications of Pumping Lemma

The applications of pumping lemma are as follows,
1. It is used to prove that, certain languages are not regular.

2. It is useful to determine whether the language accepted by finite automata is finite or infinite.

The general steps applied in pumping lemma are as follows.

1. Generate an input string (W) from the given language L .

2. Divide the length of input string (W) as u, v, x respectively and apply the conditions that are imposed by pumping lemma.

3. Verify whether they exist any contraction to pumping lemma on any value.

Example

Consider the language $L = \{W \in \{0, 1\}^* \mid \text{the number of 0's in } W \text{ is a perfect square}\}$. To prove that the given language is not regular. Let us assume that L is regular and n is pumping lemma constant. Let string $W = 0^{n^2}$. Let us now split the string (W) into three parts $W = uvw$.

If $P = 2$ then,

$$uv^2w = 0^{n^2 + |V|}$$

From pumping lemma, it is known that,

If $|uV| \leq n$ and $|v| > 0$ then,

$$n^2 \leq n^2 + |v| \leq (n+1)^2$$

Thus, $uv^2w \notin L$

$\therefore$ Language $L = \{W \in \{0, 1\}^* \mid \text{the number of 0's in } W \text{ is a perfect square}\}$ is not regular.

Q26. State pumping lemma for regular languages. Prove that the following language $\{a^n \mid n \text{ is a prime number}\}$ is not a regular. March-21(R18), Q3(b)

OR

Apply pumping lemma for the following language $L = \{a^n \mid n \text{ is prime}\}$ and prove that it is not regular. May-19(R16), Q4(b)

OR

Prove that regular set $L = \{1^p \mid p \text{ is a prime}\}$ is not regular. [Note: Assume $a = 1$ and $n = p$ and solve the problem accordingly].

Answer : Dec.-19(R16), Q5(a)

For answer refer Unit-II, Page No. 48, Q24, Topic: Pumping Lemma for Regular Languages.

Given language $L = \{a^n \mid n \text{ is prime}\}$

Assume $L = a^n$ is regular and n is prime number

Let $Z = uvw$

$$|z| = |uvw| = n$$

Consider, $z = uv^2w$ $n < |uv^2w|$

Now, add 1 to n .

$$n < n + 1$$

Since $n + 1$ is not prime number.

Hence $uv^2w \notin L$

$\therefore L$ is not regular.

Q27. Prove that the language $L = \{0^n 1^{n+1} \mid n > 0\}$ is not regular.

Answer :

Given that,

$$L = \{0^n 1^{n+1} \mid n > 0\}$$

Assume L is regular and n is the number of states in FA accepting L ,

Let, $|z| = |uvw|$

$$= 0^n 1^{n+1}$$

Length of the string $|z| = n + n + 1 = 2n + 1 > n$ i.e., string of odd length,

By pumping lemma,

$$z = |u v. v. w|$$

If $2n+1$ is added to the string, then,

$$(2n+1) < (2n+1)(2n+1)$$

$$2n+1 < 4n+2$$

Now, substitute $n = 1$ in $4n+2$,

$$z = 4n+2 = 6 \text{ (Which is even)}$$

$\neq$ Odd string

If 1 is added to the length of string $|z|$, then,

$$|z| = 2n + 1$$

$$= 2n + 2$$

$$= 2(n+1)$$

= Even length string

$\neq L$

Contradiction is obtained

$\therefore L$ is not a regular language.

Q28. Prove that $L = \{WW^* \mid W \text{ is a binary string}\}$ is not regular language.

Answer :

Sep.-21(R18), Q4(a)

Given that,

$$L = \{WW^* \mid W \text{ is a binary string}\}$$

Assume L is regular and n is the number of states in FA accepting L .

Let $Z = uvw = ababbaba$ where $|Z| \geq n$, $|uv| \leq n$ and $|v| > 0$.

By Pumping Lemma, Z is splitted into uvw .

Where, $u = a$, $v = b$ and $w = abababa$

Then, $Z = uv^i w$

$$Z = uw$$

$$Z = aabbaba$$

$$\neq ww^R$$

i.e., $Z = uv^i w \notin L$ which is a contradiction.

Let, $i = 2$;

Then, $Z = uv^2 w = abbabababa$

$$\neq ww^R$$

i.e., $Z = uv^2 w \notin L$ which is contradiction.

Since, $uv^i w \notin L$.

Therefore, $L = ww^R$ is not regular.

2.3 CONTEXT-FREE GRAMMARS

2.3.1 Definition of Context-Free Grammars

Q29. Define context-free grammar. Discuss the four important components in a grammatical description of a language.

Answer :

Context-Free Grammar (CFG)

A context free grammar (or simply a grammar) describes the programming language constructs. It consists of terminals, non-terminals, a start symbol and productions. It is denoted by $G = (V_p, V_n, S, P)$.

Examples of CFG

1. The grammar that generates strings having equal number of a 's and b 's is,

$$G = (\{S\}, \{a, b\}, P, S)$$

Where, P is,

$$S \rightarrow aSbS$$

$$S \rightarrow bSaS$$

$$S \rightarrow \epsilon$$

2. The grammar that generates the set of all strings with exactly one 1 over $\Sigma = \{0, 1\}$ is,

Where, P is,

$$S \rightarrow A1A$$

$$A \rightarrow 0A \mid \epsilon$$

Components in a Grammatical Description of a Language

Following are the four important components in a grammatical description of a language.

1. Terminals (V_t)

Terminals are finite set of symbols from which the strings for the language are formed. These are generally represented by small letters $a, b, c, \dots$

2. Non-terminals (V_n)

Non-terminals are the finite set of variables that represents a language i.e., a set of strings. These are generally represented by capital letters $A, B, C, \dots$

3. Start Symbol (S)

Start symbols represents the language being defined by the grammar. It is generally denoted by S .

4. Productions (P)

Productions are the set of rules that describes the recursive definition of a language. All productions are of the form,

$$\alpha \rightarrow \beta$$

Where,

α : A non-terminal i.e., $\alpha \in V_n$

β : A combination of terminals and non-terminals i.e., $\beta \in (V_t \cup V_n)^*$.

2.3.2 Derivations using a Grammar Leftmost and Rightmost Derivations, The Language of a Grammar

Q30. Discuss about,

- (i) Derivations
- (ii) Recursive Inference.

Answer :

(i) Derivations

Derivation is a method of replacing the non-terminal symbol present on RHS of a production rule with a terminal input. In other words the process of deriving an input string from a set of production rules is called as derivation.

Let,

$G = (V_n, V_t, P, S)$ be a context-free grammar.

A tree representation for the derivation of the given production rules for a CFG is called a derivation tree or parse tree.

A derivation tree has the following properties:

1. The root node is always the start symbol of G i.e., S .
2. The leaf nodes are always the terminals a i.e., $a \in (V_t \cup \{\epsilon\})$.
3. The interior nodes are always the non-terminals, A_i i.e., $A_i \in V_n$.
4. If the children of a node $A \in V_n$ are $X_1 X_2 \dots X_k$, then the production $A \rightarrow X_1 X_2 \dots X_k$ is in P .

Example

A CFG, $G = (V_n, V_t, P, S)$ for generating odd number of 1's is,

$$S \rightarrow aSa$$

$$S \rightarrow a$$

Q33. Discuss in brief about context free language.

Answer :

Context Free Language

The language generated by CFG is called as Context Free Language (CFL). A language 'L' is said to be CFL if and only if $L = L(G)$.

Where, G is the context free grammar.

Example: Consider grammar G as,

$$\begin{aligned} G: S &\rightarrow abB \\ A &\rightarrow aaBb \\ B &\rightarrow bbAa \\ A &\rightarrow \epsilon \end{aligned}$$

Derive the strings generated by G. Start with start symbol S.

$$\begin{aligned} S &\Rightarrow abB \\ &\Rightarrow abbbAa \\ &\Rightarrow abbbbaagb \\ &\Rightarrow abbaabbAab \\ &\Rightarrow abbaabb\epsilon ab \\ &\Rightarrow abbaabbab \end{aligned}$$

Clearly, the grammar generates strings containing n number of a 's and $(n+1)$ number of b 's

$\therefore$ The CFL is given by,

$$L = \{ab(bbaa)^n bba(ba)^n\}.$$

Sentential Forms

If $A \xRightarrow{*} \beta$, β is said to be in sentential form if β contains terminals or non-terminals. Here, the symbol $\xRightarrow{*}$ means a number of derivation steps used.

Left-sentential Form

If $A \xRightarrow{lm} \beta$ using leftmost derivation then β is a left-sentential form of the grammar.

Right-sentential Form

If $A \xRightarrow{rm} \beta$ using rightmost derivation (canonical derivation) and β may contain non-terminals, then β is a right-sentential form of the grammar.

Example: Consider the grammar,

$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid id$$

The string $w = id + id$ can be derived as follows,

$$\begin{aligned} E &\Rightarrow E + E \\ &\xRightarrow{lm} id + E \\ &\xRightarrow{lm} id + id \end{aligned}$$

Here, the strings $E + E$, $id + E$, and $id + id$ are all sentential forms, specifically, they are left-sentential forms. The right-sentential forms can be obtained as,

$$\begin{aligned} E &\Rightarrow E + E \\ &\xRightarrow{rm} E + id \\ &\xRightarrow{rm} id + id \end{aligned}$$

Here, $E + E$, $E + id$ and $id + id$ are also sentential forms, specifically they are right-sentential forms.

Q34. Obtain CFG for the following language

(i) $L = \{WW^R \mid W \text{ is in } (a+b)^*, W^R \text{ is the reversal of } W\}$

(ii) $L = \{W \mid W \text{ has a substring}\}$.

Answer :

(i) Given that,

$L = \{WW^R \mid W \text{ is in } (a,b)^*\}$ where W^R is the reversal of W . The given language generates a string over $\{a,b\}$ and can be defined as,

$$L = \{aa, bb, abba, baab, aaaa, bbbb, aabbaa, abbbba, \dots\}$$

Let the CFG for L be defined as,

$$G = (V, T, P, S)$$

Where,

$$V = A$$

$$T = \{a, b\}$$

$$S = A$$

P is defined as,

$$A \rightarrow aAa$$

$$A \rightarrow bAb$$

$$A \rightarrow \epsilon$$

Let us check, if the CFG generates strings using derivation. Consider a string $W = aabbaa$ the most derivation for W may be,

$$\begin{aligned} &\xRightarrow{LM} A \Rightarrow aAa \\ &\xRightarrow{LM} \Rightarrow aaAaa \\ &\xRightarrow{LM} \Rightarrow aabAbaa \\ &\xRightarrow{LM} \Rightarrow aab\epsilon baa \\ &\Rightarrow aabbaa \end{aligned}$$

Since, $W = aabbaa$ is derived from the CFG the CFG constructed above generates $L = \{WW^R\}$

Given that $L = \{W \mid W \text{ has a substring}\}$

(ii) The given language generates a substring over $\{a, b\}$ and can be defined as,

$$L = \{a, b, ab, ba, aaa, bbb, aba, bab, \dots\}$$

Let the CFG for L be defined as,

$$G = (V, T, P, S)$$

Where,

$$V = \{A\}$$

$$T = \{a, b\}$$

$$S = \{A\}$$

P is defined as,

$$A \rightarrow aAb$$

$$A \rightarrow aA$$

$$A \rightarrow bA$$

$$A \rightarrow \epsilon$$

Let us check if the CFG generates strings in L using derivation. Consider a substring $W = a a a$ the left most derivation for w may be,

$$\begin{aligned} A &\xRightarrow{LM} a A b \\ &\xRightarrow{LM} a a A b \\ &\Rightarrow a a a A b \\ &\Rightarrow a a a a A b \\ &\Rightarrow a a a a \epsilon b \\ &\Rightarrow a a a a b \end{aligned}$$

Since, substring $W = a a a$ is derived from CFG, the CFG, constructed for grammar generates $L = \{W|W \text{ has a substring}\}$.

Q35/ Construct the Context Free Grammar for the Language $L = \{0^{2n}1^m \mid n \geq 0, m \geq 0\}$

Answer : Feb.-22(R18), Q5(a)

Given that,

$$L = \{0^{2n}1^m \mid n \geq 0, m \geq 0\}$$

Let, $G = \{V_n, V_p, P, S\}$ be CFG for the given grammar.

Here,

$$\begin{aligned} V_n &= \{S\} \\ V_p &= \{0, 1\} \end{aligned}$$

P is defined as follows,

$$\begin{aligned} S &\rightarrow \epsilon | SS \\ S &\rightarrow 001S | 00S1 \\ S &\rightarrow S010 | S001 | S100 \\ S &\rightarrow 01S0 | 010S | 0S10 | 0S01 \\ S &\rightarrow 100S | 1S00 | 10S0 \end{aligned}$$

For example, the grammar must generate string like 001, 000011, 000000111,

Let us derive a string 000011 which is in $L(G)$

$$\begin{aligned} S &\rightarrow 00S1 \\ &\rightarrow 0000S11 \\ &\rightarrow 0000\epsilon11 \\ &\rightarrow 000011 \end{aligned}$$

Therefore, string '000011' accepted by $L(G)$

Hence, G is a CFG for given language.

Q36/ Find CFG for the language $L = \{a^i b^j c^k \mid i = j\}$.

Answer : March-21(R18), Q5(a)

Given that,

$$L = \{a^i b^j c^k \mid i = j\}$$

The given language generates the strings containing any number of a 's followed by any number of b 's and c 's. Where, number of a 's are equal to number of b 's.

Let the CFG be, $G = \{V_n, V_p, P, S\}$

Here, $V_n = \{S, A, B\}$

$$V_p = \{a, b, c\}$$

$$S = \{S\}$$

The productions are given by,

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aAb \mid \epsilon \\ B &\rightarrow cB \mid \epsilon \end{aligned}$$

Let us derive the string $aabbccccc$ from the grammar G .

Let us take,

$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aAB B & [\because A \rightarrow aAb] \\ &\rightarrow aaAB B B & [\because A \rightarrow aAb] \\ &\rightarrow aa\epsilon BB B & [\because A \rightarrow \epsilon] \\ &\rightarrow aaBB c B & [\because B \rightarrow cB] \\ &\rightarrow aaBB cc B & [\because B \rightarrow cB] \\ &\rightarrow aaBB ccc B & [\because B \rightarrow cB] \\ &\rightarrow aaBB cccc B & [\because B \rightarrow cB] \\ &\rightarrow aaBB cccc \epsilon & [\because B \rightarrow \epsilon] \\ &\rightarrow aaBB cccc & (\text{Accepted}) \end{aligned}$$

Since the string $aabbccccc$ is derivable from G . Therefore, G is a CFG for a given language.

2.3.3 Parse Trees

Q37. Define parse trees and yield of a parse tree.

OR

Discuss in detail about leftmost and rightmost derivation tree with example.

(Refer Only Topics: Leftmost Derivation Tree and Rightmost Derivation Tree)

Answer :

Dec.-19(R16), Q7(b)

Derivation Tree

A derivation tree or parse tree shows a graphical representation of a derivation. It is a labelled tree in which interior nodes are labelled by non-terminals and the leaves of a node are labelled by terminals or non-terminals. A parse tree for a context free grammar G has the following properties.

1. The root of a parse tree is the start symbol of the CFG (G).
2. The interior node of a parse tree are always the non-terminal.
3. Every leaf of a parse tree is labeled with either a terminal, variable or ϵ .
4. If the grammar G has a production of form $A \rightarrow y_1 y_2 y_3 \dots y_n$ and the interior node of a parse tree contains the non-terminal A then its children must be labelled by $y_1, y_2, \dots, y_n$ from left to right.

5. If the grammar G has a production of form $A \rightarrow \epsilon$ and if the interior node of a parse tree contains the non-terminal A then ϵ must be the only child of A .

Derivation trees are of two types

- Left most derivation tree
- Rightmost derivation tree.

(i) Leftmost Derivation Tree

In left most derivation tree, the leftmost non-terminal is replaced first with a terminal symbol. The leaves of a parse tree contains terminals or ϵ which when read from left to right forms a string that is derived from the root node.

Example

Consider the context free grammar for arithmetic expressions.

$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid id$$

Let the string $(w) = -(id + id)$

The leftmost derivation for the string $-(id + id)$

is,

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(id + E) \Rightarrow -(id + id)$$

and the corresponding parse tree is shown below.

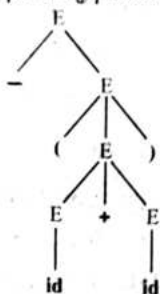
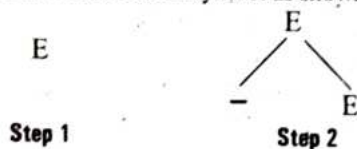


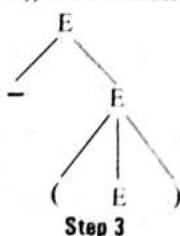
Figure: Parse Tree for String $-(id + id)$

The parse tree for $-(id + id)$ is constructed using the following steps.

- The root node of a tree is the start symbol of G , that is E .
- To apply the production, $E \rightarrow -E$, two children – and E are added to start symbol as shown below.

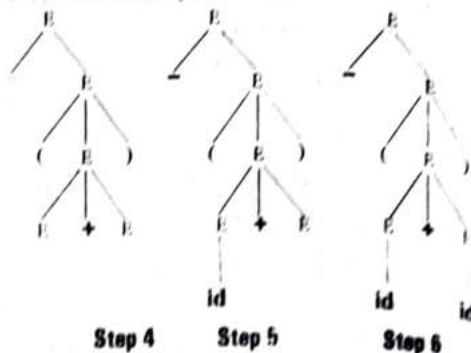


- The non-terminal E is further expanded using the production $E \rightarrow (E)$. By adding three children to E as $((E$ and $))$ as shown below.



Step 3

The successive steps in the construction of parse tree are shown below,



Step 4

Step 5

Step 6

(ii) Rightmost Derivation Tree

In rightmost derivation tree the rightmost non-terminal is replaced first with a terminal symbol.

Example

Consider the context free grammar for arithmetic expressions,

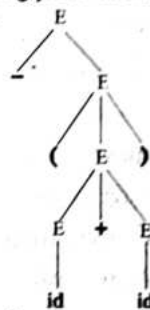
$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid id$$

Let the string $(w) = -(id + id)$

The rightmost derivation for the string $-(id + id)$ is,

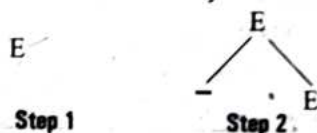
$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + id) \Rightarrow -(id + id)$$

and the corresponding parse tree is shown below,



The parse tree for $-(id + id)$ is constructed using the following steps,

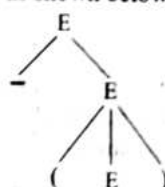
- The root node of a tree is the start symbol of G , that is E .
- To apply the production, $E \rightarrow -E$, two children – and E are added to start symbol as shown below.



Step 1

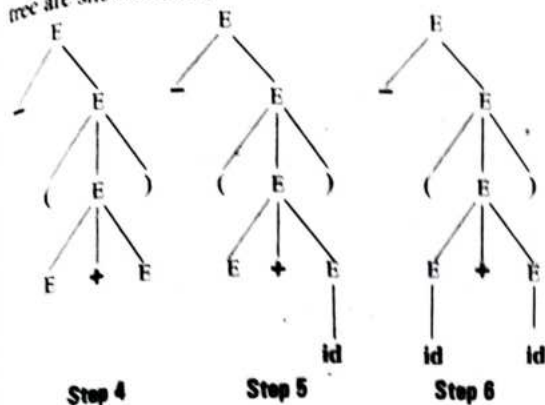
Step 2

- The non-terminal E is further expanded using the production $E \rightarrow (E)$. By adding the children to E as $(E$ and $))$ as shown below.



Step 3

The successive steps in the construction of parse tree are shown below.



Uses of Derivation Tree

1. The uses of derivation tree are as follows,
1. It is a data structure that gives hierarchical representation of a given sentence.
2. It is used to check whether a grammar is ambiguous (or) unambiguous.
3. It is used for checking whether a string is accepted by a given CFL (or) not.
4. It is used for deriving leftmost derivation of a given sentence.
5. It is used for deriving rightmost derivation of a given sentence.

Yield of a Parse Tree

The yield of a parse tree can be defined as a string which is obtained from the root variable by concatenating the leaves from left side of a parse tree. Generally, the parse tree satisfying the following conditions are considered to be more important.

- (a) Parse trees whose leaves are named either with a terminal or ϵ
- (b) Parse trees whose root node is denoted by a start symbol.

Example

Consider the parse tree for the grammar $S \rightarrow aSb$ such that satisfies the language $L = \{a^n b^n \text{ such that } n > 1\}$

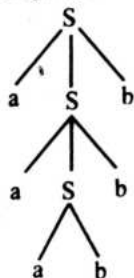


Figure: Parse Tree for the above Grammar

The leaves from left to right when concatenated give the string $a^3 b^3$. This string is called the yield of a parse tree.

Q38. Show how trees are constructed from inferences.

Answer :

Construction of Tree from Inferences

Theorem

Consider a CFG, $G = (V, T, P, S)$ such that the recursive inference procedure defines the terminal string w in the language of variable A . Then, there is a parse tree with root A and yield w .

Proof

It is based on the induction which involves some steps to show that w satisfies the language of A .

Basic Case

Let $G = (V, T, P, S)$ be a CFG where $A \rightarrow w$ is a production containing one leaf for every position of w . The following figure illustrates construction of a parse tree in the basic case.

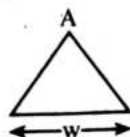


Figure: Parse Tree for the Basic Case

Induction Hypothesis

Assume that $n + 1$ steps are required to show 'w' satisfies the language of A . These steps employ few productions for A (i.e.), $A \rightarrow X_1 X_2 \dots X_k$, where X_i can be either a terminal or a non-terminal.

Now, 'w' can be divided into $w_1 w_2 w_3 \dots w_k$, such that,

- (i) When X_i is a terminal then $w_i = X_i$
- (ii) When X_i is a non-terminal or variable then there exists a string w_i which had previously occurred in the language of X_i .

Thus, on application of inductive hypothesis to w_i and x_i it can be concluded that a parse tree occurs with yield w_i and root x_i . Therefore, the generated parse tree can be represented as follows.

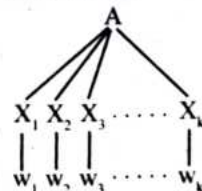


Figure: Parse Tree

Q39. Show how derivations are constructed from trees.

Answer :

Construction of Derivations from Trees

Theorem

Let a CFG, $G = (V, T, P, S)$ and consider a parse tree with a root A and yield w . Then, there exist a leftmost derivation $A \xrightarrow{lm} w$ in CFG, G .

Proof

To perform induction on height of the parse tree.

Basic Case

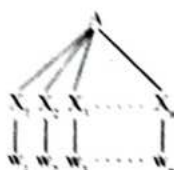
Least heighted tree is illustrated as shown below.



Since, the above illustrated tree is a parse tree $A \rightarrow w$ must be a production such that there exists a leftmost derivation of w from A (i.e., $A \xRightarrow{*}_{lm} w$).

Induction Hypothesis

Consider the height of the parse tree as n and root node as A containing the child nodes labeled as $X_1, X_2, X_3, \dots, X_n$.



In this case, child nodes are further divided as

$$w = w_1 w_2 w_3 \dots w_{i-1} w_i w_{i+1} \dots w_n$$

Where,

- $w_i = X_i$ when $X_i \in \text{terminal}$
- $X_i \xRightarrow{*}_{lm} w_i$ is in G when $X_i \in \text{variable}$, also the height of the parse tree is less than n and X_i forms the root of a subtree.

The induction hypothesis is made again to construct left most derivation for w_i .

Now consider,

$$A \xRightarrow{*}_{lm} w_1 w_2 w_3 \dots w_{i-1} X_i X_{i+1} \dots X_n$$

This assumption involves the following two cases,

- If $X_i \in T$, then $A \xRightarrow{*}_{lm} w_1 w_2 \dots w_{i-1} X_i X_{i+1} \dots X_n$. The derivation completes here since, $X_i = w_i$.
- If $X_i \in V$, then $X_i \xRightarrow{*}_{lm} \alpha_1 \xRightarrow{*}_{lm} \alpha_2 \xRightarrow{*}_{lm} \dots \xRightarrow{*}_{lm} w_i$.

The derivation has to be continued as shown below,

$$\begin{aligned} w_1 w_2 w_3 \dots w_{i-1} X_i X_{i+1} \dots X_n &\xRightarrow{*}_{lm} \\ w_1 w_2 w_3 \dots w_{i-1} \alpha_1 X_{i+1} \dots X_n &\xRightarrow{*}_{lm} \\ w_1 w_2 w_3 \dots w_{i-1} \alpha_2 X_{i+1} \dots X_n &\xRightarrow{*}_{lm} \\ w_1 w_2 w_3 \dots w_{i-1} \alpha_3 X_{i+1} \dots X_n &\xRightarrow{*}_{lm} \\ &\vdots \\ w_1 w_2 w_3 \dots w_{i-1} w_i X_{i+1} \dots X_n &\xRightarrow{*}_{lm} \end{aligned}$$

$$\therefore A \xRightarrow{*}_{lm} w_1 w_2 w_3 \dots w_{i-1} w_i X_{i+1} \dots X_n$$

Finally, the leftmost derivation of w_i is obtained from A where $i = n$.

Q40. Let G be the grammar

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid aS \mid bAA \mid$$

$$B \rightarrow b \mid bS \mid aBB$$

For the string $abbaab$ find leftmost and rightmost derivations, and parse tree. Find the language accepted by the grammar.

Answer :

Given grammar is,

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid aS \mid bAA \mid$$

$$B \rightarrow b \mid bS \mid aBB$$

Leftmost Derivation

$$S \Rightarrow aB$$

$$\Rightarrow abS$$

$$\Rightarrow abba$$

$$\Rightarrow abbaaS$$

$$\Rightarrow abbaaBB$$

$$\Rightarrow abbaab$$

$\therefore S \Rightarrow aB$
 $\therefore A \rightarrow aS$
 $\therefore S \rightarrow aB$
 $\therefore B \rightarrow b$
(Accepted)

Rightmost Derivation

$$S \Rightarrow aB$$

$$\Rightarrow abS$$

$$\Rightarrow abba$$

$$\Rightarrow abbaaS$$

$$\Rightarrow abbaaBB$$

$$\Rightarrow abbaab$$

$\therefore S \rightarrow aB$
 $\therefore B \rightarrow bS$
 $\therefore S \rightarrow bA$
 $\therefore A \rightarrow aS$
 $\therefore S \rightarrow aB$
 $\therefore B \rightarrow b$
(Accepted)

Parse Tree

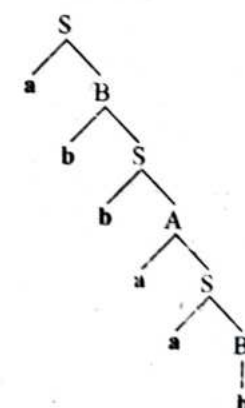


Figure: Left and Right Parse Tree for $abbaab$

Here, Parse tree of both leftmost and rightmost derivation for $abbaab$ is same.

Since, the given string " $abbaab$ " can be derived by the CFG and it contains equal number of a 's and b 's. So, the language accepted by the CFG is $L = \{a^n b^n \mid n \geq 1\}$.

2.3.4 Ambiguity in Grammars and Languages

Q41. Explain about ambiguity in grammars and languages with example.

Dec.-19(R16), Q7(a)

Answer :

Ambiguous Grammar

A grammar is said to be ambiguous, if it produces more than one derivation tree for some input string generated by it. In other words, if there are more than one leftmost derivation or more than one rightmost derivation for a given string. Then such a grammar is said to be ambiguous.

An ambiguous grammar contains productions of the form,

$$S \rightarrow S \alpha S$$

Where, α may be string of terminals or non-terminals i.e., the same non-terminal appears twice on R.H.S of a production.

Example

Consider the following grammar,

$$S \rightarrow SS | a | b$$

Let us derive a string from this grammar,

$$S \Rightarrow SS$$

$$\Rightarrow SSS$$

$$\Rightarrow aSS$$

$$\Rightarrow abS$$

$$\Rightarrow aba$$

Now, construct derivation trees for the string *aba*.

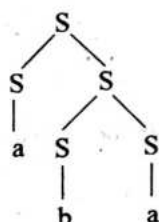


Figure (a): Leftmost Derivation

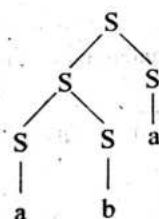


Figure (b): Rightmost Derivation Tree

Since there are two derivation trees for the string *aba*, the given grammar is ambiguous.

A context free language wherein all the grammars are ambiguous is known as inherent ambiguity.

Example

For instance the language, $L = \{p^m q^n r^n s^m / m, n \geq 1\} \cup \{p^m q^n r^n s^m / n, m \geq 1\}$ posses inherent ambiguity. This is because there exist number of strings in the $p^n q^n r^n s^n$ where $n > 1$ having two different left most derivations.

The grammar for the above language is given below.

$$S \rightarrow A | BC$$

$$A \rightarrow aAd | aDd$$

$$D \rightarrow bDc | bc$$

$$B \rightarrow aBb | ab$$

$$C \rightarrow cCd | cd$$

The string $a^2 b^2 c^2 d^2$ belonging to the above grammar can be derived in the following two LMDs.

LMD₁

$$S \Rightarrow A$$

$$\Rightarrow aAd$$

$$\Rightarrow aaDdd$$

$$\Rightarrow aabDcdd$$

$$\Rightarrow aabbccdd$$

LMD₂

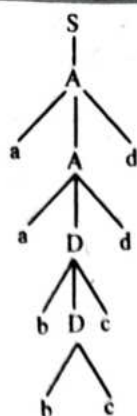
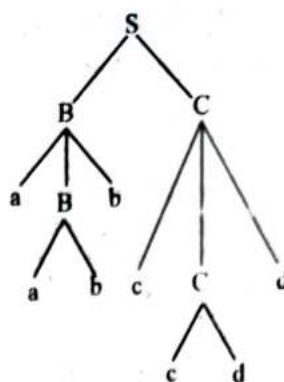
$$S \Rightarrow BC$$

$$\Rightarrow aBbC$$

$$\Rightarrow aabbC$$

$$\Rightarrow aabbccCd$$

$$\Rightarrow aabbccdd$$

Figure: Parse Tree for LMD₁Figure: Parse Tree for LMD₂

Q42. Explain in detail how the ambiguity is removed from the grammar and how it can be expressed through left most derivation.

Answer :

Removing Ambiguity from a Grammar

If a grammar is of form

$$A \rightarrow \theta_1 A \lambda_1 \beta \mid \theta_1 \mid \theta_2 \mid \theta_3 \mid \dots \mid \theta_n$$

Then the new unambiguous grammar can be created by introducing a new variable called A' and converting the above grammar to form,

$$A \rightarrow \theta_1 A \lambda_1 \beta \mid A'$$

$$A' \rightarrow \theta_1 \mid \theta_2 \mid \theta_3 \mid \theta_4 \mid \dots \mid \theta_n$$

Here, A' is the new variable which eliminate the ambiguity from grammar G .

Example

Consider the following ambiguous grammar,

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

To make the above grammar unambiguous a new variable E' is introduced such that,

$$E \rightarrow E + E' \mid E'$$

$$E \rightarrow E * E \mid (E) \mid id$$

However, still the production $E' \rightarrow E * E$ retains ambiguity. Thus, remaining ambiguity can be removed by substituting,

$$E \rightarrow E * E' \mid (E) \mid id$$

Hence, on removing the remaining ambiguity another variable called F is introduced, such that,

$$E \rightarrow E * F \mid F$$

$$F \rightarrow (E) \mid id$$

$\therefore$ The new unambiguous grammar is,

$$E \rightarrow E + E' \mid E$$

$$E \rightarrow E * F \mid F$$

$$F \rightarrow (E) \mid id$$

Expressing Ambiguity Through Left Most Derivations

The ambiguity is said to be expressed through Left Most Derivations (LMDs) if there exist two distinct LMDs even if the grammar is unambiguous.

Example

Consider the grammar,

$$E \rightarrow I$$

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$I \rightarrow a$$

This grammar possesses two different derivations as shown below.

LMD ₁	LMD ₂
$E \Rightarrow E + E$	$S \Rightarrow E * E$
$\Rightarrow 1 + E$	$\Rightarrow E + E * E$
$\Rightarrow a + E$	$\Rightarrow 1 + E * E$
$\Rightarrow a + E * E$	$\Rightarrow a + 1 * E$
$\Rightarrow a + 1 * E$	$\Rightarrow a + a * 1$
$\Rightarrow a + a * E$	$\Rightarrow a + a * a$
$\Rightarrow a + a * 1$	
$\Rightarrow a + a * a$	

Therefore, it is clear that the ambiguity of the grammar can be expressed through LMDs.

Q43. Prove that the following grammar of arithmetic expression is ambiguous.

$$E \rightarrow E + E | E * E | (E) | id$$

Answer :

A CFG is said to be ambiguous, if it produces more than one derivation tree for some input string generated by it.

Given grammar is,

$$E \rightarrow E + E | E * E | (E) | id$$

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow id$$

A CFG is generally given by $G = (V, T, P, S)$

Here,

$$V = \{E\}$$

$$T = \{id\}$$

$$P \rightarrow \{E \rightarrow E + E$$

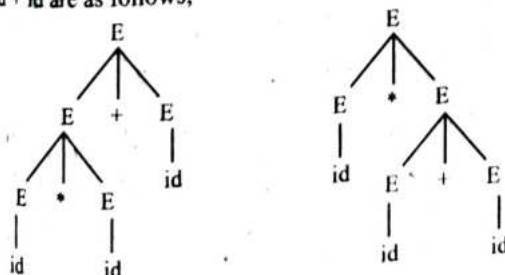
$$E \rightarrow E * E$$

$$E \rightarrow E$$

$$E \rightarrow id\}$$

$$S \rightarrow \{E\}$$

The two different parse trees for the string $id * id + id$ are as follows,



Therefore, the given grammar is an ambiguous one.

Q44. Show that the following grammars are ambiguous.

(i) $\{ S \rightarrow aSbS | bSaS | \lambda \}$ and

(ii) $\{ S \rightarrow AB | aaB, A \rightarrow a / Aa, B \rightarrow b \}$

OR

Show that the following grammar is ambiguous or not.

$$S \rightarrow AB | aaB,$$

$$A \rightarrow a | Aa,$$

$$B \rightarrow b$$

(Refer Only Topic: Topic: (ii) $\{ S \rightarrow AB | aaB, A \rightarrow a | Aa, B \rightarrow b \}$)

Answer :

Feb.-22(R18), Q6(b)

(i) $\{ S \rightarrow aSbS | bSaS | \lambda \}$

Given grammar G is,

$$S \rightarrow aSbS | bSaS | \lambda$$

In the given grammar G ,

$$G = \{V_n, V_t, P, S\}$$

Where, $V_n = \{S\}$

$$V_t = \{a, b, \lambda\}$$

$$P = \{$$

$$S \rightarrow aSbS$$

$$S \rightarrow bSaS$$

$$S \rightarrow \lambda$$

$$\}$$

$$S = \{S\}$$

To check whether a given grammar ' G ' is ambiguous (or) not, let us construct the derivation trees for a string $L(G)$.

Initially, let us derive a string from the above grammar.

$$\begin{aligned} S &\Rightarrow aSbS && [\because S \rightarrow aSbS \text{ Where, 'S' is a start symbol}] \\ &\Rightarrow abSaSbS && [\because S \rightarrow bSaS] \\ &\Rightarrow ab\lambda aSbS && [\because S \rightarrow \lambda] \\ &\Rightarrow abaSbS && [\text{Consider the leftmost nonterminal S}] \\ &\Rightarrow aba\lambda bS && [\because S \rightarrow \lambda] \\ &\Rightarrow ababS && [\text{Consider the leftmost nonterminal S}] \\ &\Rightarrow abab\lambda && [\because S \rightarrow \lambda] \\ &\Rightarrow abab && (\text{Accepted}) \end{aligned}$$

Now, construct derivation trees for the string $W = abab$. i.e., both leftmost derivation and rightmost derivation trees,

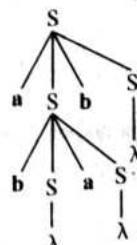


Figure (1): Leftmost Derivation of abab

VERY SHORT QUESTIONS WITH SOLUTIONS (VSQS)

Q1. Define regular expression.

Answer :
A regular expression is a concise notation for denoting regular sets. Regular expressions describe the language accepted by a finite automata.

Q2. List any two identity rules of regular expressions.

Answer :
The identity rules of regular expressions are given below,

1. $\phi + R = R$
2. $\epsilon + R^* = R^*$

Q3. Write any two properties of regular languages.

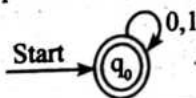
Jan./Feb.-23(R18), Q1(e)

Answer :

1. A regular language is accepted by NFA, DFA and read-only by TM.
2. A regular language is described by regular expression.
3. A regular language generates a regular grammar.

Q4. Design a FA for RE = $(0^*1^*)^*$.

Answer :
Given regular expression is RE = $(0^*1^*)^*$
The finite automata for given regular expression is constructed as follows,



Q5. Construct a DFA for Regular language consisting of any number of a's and b's.

April-18(R16), Q4(b)

Answer :
DFA for the language containing any number of a's and b's is,

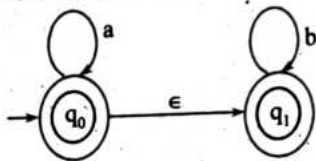


Figure: DFA

Q6. State Pumping lemma for regular languages.

Answer :
Let,
 $M = (Q, \Sigma, \delta, q_0, F)$ be a finite state machine with n states. Let L be a regular language accepted by M .

Then a string $Z \in L$ such that $|Z| \geq n$ can be written as $Z = uvw$ by satisfying the following conditions,

- (i) $v \neq \epsilon$
- (ii) $|uv| \leq n$ and
- (iii) The string $uv^i w \in L$, for all $i \geq 0$

The theorem states that given a string Z belonging to L always find a substring V near the beginning of Z that can be pumped or repeated as many times as users like or deleting it, places the resulting string in the language.

Q7. Define context free grammar.

Answer :

A Context Free Grammar (CFG) (or) a grammar describes the programming language constructs. It consists of, non-terminals, terminals start symbol and productions. It is denoted by $G = (V_n, V_t, S, P)$.

Q8. What do you mean by parse tree?

Answer :

A tree representation for the derivation of the given production rules for a CFG $G = (V_n, V_t, P, S)$ is called a derivation tree or parse tree.

Q9. Define ambiguity in CFG.

Answer :

A grammar is said to be ambiguous, if it produces more than one derivation tree for some input string generated by it.

FREQUENTLY ASKED QUESTIONS & IMPORTANT QUESTIONS

Q1. Define Regular expression. Explain about the properties of regular expressions.

Answer :

(Dec.-18(R16), Q4(a) | April-18(R16), Q4(a))



For answer refer Unit-II, Page No. 32, Q1.

Q2. Prove "If L is accepted by DFA, then there exists an equivalent regular expression which develops L".

Answer :

Important Question

For answer refer Unit-II, Page No. 35, Q7.

Q3. Discuss the method for conversion of regular expression to finite automata.

Answer :

Important Question

For answer refer Unit-II, Page No. 36, Q8.

Q4. Construct a DFA for the regular expression $(0 + 1)^*(00 + 11)(0+1)^*$.

Answer :

(Dec.-18(R16), Q4(b) | April-18(R16), Q5(a))



For answer refer Unit-II, Page No. 37, Q9.

Q5. Convert regular expression $(01^* + 1)$ to finite automata.

Answer :

(Important Question | Dec.-19(R16), Q4)

For answer refer Unit-II, Page No. 39, Q11.

Q6. Construct a NFA equivalent to the regular expression $10(0 + 11)0^*1$.

Answer :

(Important Question | Jan./Feb.-23(R18), Q4(b))

For answer refer Unit-II, Page No. 41, Q14.

Q7. Discuss the applications of a regular expressions.

Answer :

Important Question

For answer refer Unit-II, Page No. 42, Q16.

Q8. Write the steps to construct regular expression from given DFA.

(Important Question | Jan./Feb.-23(R18), Q4(b))

Answer :

Important Question

For answer refer Unit-II, Page No. 45, Q21

Q10. Construct the regular expression corresponding to the language accepted by following DFA.

Answer :

(Important Question | May-19(R16), Q5)

For answer refer Unit-II, Page No. 47, Q23.

Q11. Explain about pumping lemma.

Answer :

(Important Question | Dec.-19(R16), Q5(b))

For answer refer Unit-II, Page No. 48, Q24.

Q12. Prove that $L = \{WW^* \mid W \text{ is a binary string}\}$ is not regular language.

Answer :

(Important Question | Sep.-21(R18), Q4(a))

For answer refer Unit-II, Page No. 49, Q28.

Q13. Write about Leftmost derivation and rightmost derivation with example.

Answer :

(Jan./Feb.-23(R18), Q1(f) | May-19(R16), Q6(a))

For answer refer Unit-II, Page No. 51, Q31.

Q14. Construct a Left most Derivation for the string 0011000 using the grammar

$S \rightarrow A0S/0SS, A \rightarrow S1A/10?$

Answer :

(Important Question | Jan./Feb.-23(R18), Q9(a))

For answer refer Unit-II, Page No. 51, Q32.

Q15. Construct the Context Free Grammar for the Language $L = \{0^{2n}1^m \mid n \geq 0, m \geq 0\}$

Answer :

(Important Question | Feb.-22(R18), Q5(a))

For answer refer Unit-II, Page No. 53, Q35.

Q16. Define parse trees and yield of a parse tree.

Answer :

(Important Question | Dec.-19(R16), Q7(b))

For answer refer Unit-II, Page No. 53, Q37.

Q17. Explain about ambiguity in grammars and languages with example.

Answer :

(Important Question | Dec.-19(R16), Q7(a))

For answer refer Unit-II, Page No. 57, Q41.

Q18. Define Ambiguous Grammar? Check whether the grammar

$S \rightarrow aAB, A \rightarrow bC/cd, C \rightarrow cd, B \rightarrow c/d$

is Ambiguous or not?

Answer :

(Important Question | Jan./Feb.-23(R18), Q8(e))

For answer refer Unit-II, Page No. 60, Q47.

